



Chest Pneumonia Detection Web Application

Submitted by:

#	ID	Name
1	42110213	Omar Reda Sayed
2	42120004	Nahed Mortada Ali
3	42110294	Aya Saleh Almasry
4	42110043	Yousef Saad Mohamed
5	42110306	Esraa Eid Ahmed

A dissertation submitted in partial fulfillment of the requirements for the degree
of Bachelor of computer science and information technology

Supervised by:

Dr. Belal Badawy Amin

Egypt 2025

Committee Report

We certify we have read this graduation project report as examining committee, examine the student in its content and that in our opinion it its adequate as a project document for “Chest Pneumonia Detection Web Application”.

Chairman:

Name:

Signature:

Date: 17/06/2025

Examiner:

Name:

Signature:

Date: 17/06/2025

Supervisor:

Name: *Dr. Belal Badawy Amin*

Signature:

Date: 17/06/2025

Intellectual Property Right Declaration

This is to declare that the work under the supervision of Dr. Belal Badawy having title “Chest Pneumonia Detection Web Application” carried out in partial fulfillment of the requirements of Bachelor of Science in Computer Science is the sole property of Ahram Canadian University and the respective supervisor. It is protected under the intellectual property right laws and conventions. It can only be considered/ used for purposes like extension for further enhancement, product development, adoption for commercial/organizational usage, etc. with the permission of the University and respective supervisor. This above statement applies to all students and faculty members.

Names:

Omar Reda Sayed

Nahed Mortada Ali

Aya Saleh Almasry

Yousef Saad Mohamed

Esraa Eid Ahmed

Supervisor: Dr. Belal Badawy Amin

Anti-Plagiarism Declaration

This is to declare that the above publication produced under the supervision of Dr. Belal Badawy having title "Chest Pneumonia Detection Web Application " is the sole contribution of the author(s) and no part hereof has been reproduced illegally (cut and paste) which can be considered as Plagiarism. All referenced parts have been used to argue the idea and have been cited properly. We will be responsible and liable for any consequence if violation of this declaration is proven.

Names:

Omar Reda Sayed

Nahed Mortada Ali

Aya Saleh Almasry

Yousef Saad Mohamed

Esraa Eid Ahmed

Table of Contents

Table of contents

List of figures

List of tables

Abstract

List of abbreviations and acronyms

Chapter 1: introduction

1.1 Overview	12
1.2 Motivation	12
1.3 Objective	13
1.4 Aim	13
1.5 Scope	14
1.6 General constraints.....	14

Chapter 2 Background and Previous work

2.1 Background.....	17
2.2 Previous work.....	22

Chapter 3 planning and analysis

3.1 Planning.....	27
3.1.1 Feasibility study and estimated cost	30
3.1.2 Gantt chart	31
3.2 Analysis and limitations of existing system.....	32
3.3 Need for new system.....	34
3.4 Analysis of new system.....	35
3.4.1 User Requirement.....	35
3.4.2 System Requirements.....	36
3.4.3 Domain Requirements	37
3.4.4 Functional Requirements.....	39
3.4.5 Non- Functional Requirements.....	44
3.5 Advantages of new system.....	46
3.6 User characteristics.....	47

Chapter 4: design

4.1 Design and Implementation Constraints.....	52
4.2 Assumptions and dependencies.....	54
4.3 Risks and risk management.....	55
4.4 Design of database ERD.....	57
4.4.1 Entity Relationship Diagram.....	57
4.4.2 Mapping of Entity Relationship Diagram	57
4.5 Class diagram.....	58
4.6 Use case diagram.....	59
4.6.1 Primary Use-case Diagram.....	59
4.6.2 Use-case Scenarios.....	60
4.7 Activity diagram.....	61
4.8 Sequence diagram.....	64

4.9 State diagram.....	65
Chapter 5: Implementation	
5.1 Software Architecture.....	67
5.2 User Interface.....	80
5.3 Results and Discussion.....	84
Chapter 6: Testing	
6.1 Unit Testing	91
6.2 Integration Testing.....	99
Conclusion.....	100
Future work.....	101
References.....	102

List of Figures

Figure1: calendar-based Gantt chart	31
Figure2: Entity Relationship Diagram	57
Figure3: Mapping of Entity Relationship Diagram.....	57
Figure4: Class Diagram	58
Figure5: Use case diagram	59
Figure6: Activity Diagram	61
Figure7: Activity Diagram	62
Figure8: Activity Diagram	63
Figure9: Sequence Diagram	64
Figure10: State Diagram	65
Figure11: GoogleNet Architecture	69
Figure12: ResNet-18 Architecture	70
Figure13: DenseNet-121 Architecture	71
Figure14: Ensemble Model Architecture	72
Figure15: CNN Architecture	77
Figure16: Clean Architecture by Robert C. Martin (Uncle Bob)	78
Figure17: Web application architecture	79
Figure18: Home Page screen	80
Figure19: Sign-up Page	80
Figure20: Sign-in Page	81
Figure21: About Our System	81
Figure22: Technology Stack	82
Figure23: About AI Models Screen	82
Figure24: Chest X-ray analysis screen	83
Figure25: Development Team screen	83
Figure26: Classification Report of CNN Model	85

Figure27: Confusion Matrix of CNN Model	85
Figure28: classification report of ensemble model	86
Figure29: Confusion matrix of ensemble model	86
Figure30: Training and Validation Accuracy and Loss "CNN"	87
Figure31: Training and Validation Accuracy and Loss"ensemble"	88
Figure32: Doctor service tests in .NET API	92
Figure33: Token service tests in .NET API	93
Figure34: Authentication service tests in .NET AP	94
Figure35: Testing a Viral pneumonia diagnosed chest X-ray in FastAPI	94
Figure36: Testing a Normal diagnosed chest X-ray in FastAPI	95
Figure37: Testing a Normal diagnosed chest X-ray in fastAPI	95
Figure38: Testing a Lung-Opacity diagnosed chest X-ray in fastAPI	95

List of Tables

Table 1: Estimated Costs Breakdown	30
Table 2: Non-Functional Requirements (Performance)	44
Table 3: Non-Functional Requirements (Scalability)	44
Table 4: Non-Functional Requirements (Usability)	44
Table 5: Non-Functional Requirements (Availability)	45
Table 6: Non-Functional Requirements (Security)	45
Table 7: Non-Functional Requirements (Maintainability)	45
Table 8: Non-Functional Requirements (Compliance)	46
Table 9: User Requirements – Doctors	47
Table 10: User Requirements – Patients	48
Table 11: User Requirements – Researchers	48
Table 12: User Requirements – Medical Students	49
Table 13: User Requirements – Medical Technicians	50
Table 14: Verify Sign-up of Users	97
Table 15: Verify Log-in of Users	98
Table 16: Verify The Image Diagnosis	98
Table 17: Verify Log-Out of Users	+99

Abstract

Chest Pneumonia Detection is an AI-driven **web application** developed to assist in the early diagnosis of **pneumonia** through the classification of **chest X-ray images** using deep learning techniques. The application employs a **Convolutional Neural Network (CNN)** trained on the **COVID-19 Radiography Database** available on Kaggle, which includes thousands of labeled X-ray images across categories such as **normal**, **pneumonia**, **lung-opacity** and **COVID-19**.

The CNN model has been fine-tuned through preprocessing techniques like resizing, normalization, and data augmentation to enhance its accuracy and generalization. This enables the system to provide fast and reliable classification results, supporting medical practitioners and users in identifying pneumonia cases without the need for advanced radiological infrastructure.

As a **web-based application**, Chest Pneumonia Detection offers platform-independent access, allowing users to upload chest X-ray images and instantly receive diagnostic predictions. Built with a focus on usability and accessibility, the system is particularly beneficial for **remote or underserved regions** where expert radiologists and diagnostic tools may be unavailable.

By integrating **deep learning** with the **COVID-19 Radiography Database** and delivering it through a web platform, the application demonstrates the potential of artificial intelligence to improve **diagnostic efficiency**, enhance **healthcare accessibility**, and support **timely clinical decision-making** in the detection of pneumonia and related conditions.

Chapter 1: introduction

Chapter 1: introduction

1.1 Overview

In recent years, the integration of **artificial intelligence (AI)** in healthcare has transformed the way medical diagnostics are approached—enhancing accuracy, speed, and accessibility. **Chest Pneumonia Detection** is a web-based diagnostic tool designed to support the early identification of **pneumonia** through the analysis of **chest X-ray images** using **deep learning**.

The application leverages a **Convolutional Neural Network (CNN)** trained on the publicly available **COVID-19 Radiography Database** from Kaggle, which contains thousands of chest X-ray images labeled as **normal**, **pneumonia**, and **COVID-19**. This model enables the system to deliver fast, reliable classification results, helping healthcare providers and patients make informed decisions at the earliest possible stage.

Built as a **web application**, the platform offers easy and broad access across devices without requiring installation. Users can upload chest X-rays through a simple interface and receive near-instant feedback. The backend is powered by deep learning frameworks, while data management and security are handled using modern web technologies and cloud infrastructure.

This project addresses critical healthcare needs by offering a **scalable**, **cost-effective**, and **accessible** solution for pneumonia detection, especially in **resource-limited settings**. By combining AI-driven diagnostics with the flexibility of web deployment, **Chest Pneumonia Detection** demonstrates how technology can play a vital role in improving public health outcomes.

1.2 Motivation

The motivation for this graduation project is to address the critical challenge of accurately diagnosing pneumonia, using the Chest X-Ray Images (Pneumonia) dataset from Kaggle by Paul Timothy Mooney. Pneumonia is a severe respiratory infection that remains a leading cause of mortality, particularly among children and the elderly. Traditional diagnostic methods using chest X-rays require expert interpretation and can be time-consuming, leading to delays in treatment and higher risk of complications.

By leveraging advanced deep learning techniques such as Convolutional Neural Networks (CNN) and Support Vector Machines (SVM), this project aims to develop an effective model capable of classifying chest X-ray images with high accuracy. The goal is to enhance the diagnostic process, enabling rapid differentiation between bacterial and viral pneumonia. This can significantly improve treatment outcomes and reduce pneumonia-related mortality.

Additionally, this research aligns with the Sustainable Development Goals (SDGs), particularly Goal 3 (Good Health and Well-being), by contributing to better healthcare solutions through innovative technology.

1.3 Objectives

- Develop a Deep Learning Classification Model:

Design a model based on Convolutional Neural Networks (CNN) to classify chest X-ray images into three categories: normal lungs, bacterial pneumonia, and viral pneumonia.

- Compare Performance of Different Models:

Evaluate and compare the performance of CNN and Support Vector Machines (SVM) in classifying chest X-ray images to identify the most effective and accurate model for pneumonia diagnosis.

- Improve Diagnostic Accuracy:

Utilize data augmentation and hyperparameter tuning techniques to enhance the model's performance, aiming to achieve higher accuracy compared to traditional diagnostic methods.

- Develop a Practical Diagnostic Tool:

Create a model that can be practically implemented in clinical settings, assisting healthcare professionals in quickly identifying the type of pneumonia and improving treatment decisions.

- Support Innovation in Healthcare:

Contribute to the advancement of AI-driven solutions in medical diagnostics, aligning with global health initiatives to enhance the quality of care and support the Sustainable Development Goals related to health and innovation.

- System speeds up detection:

enabling timely treatment and reducing mortality rates in vulnerable groups like children and the elderly. By automating diagnosis, it minimizes workload for healthcare professionals and improves accessibility.

This initiative also addresses challenges like class imbalance in datasets and noisy images, contributing to research in AI-powered medical solutions.

1.4 Aim

This project aims to develop an accessible, AI-powered **web application** capable of analyzing **chest X-ray images** for the **early detection of pneumonia**. By leveraging widely available internet-connected devices and a lightweight web-based interface, **Chest Pneumonia Detection** delivers a low-cost, scalable solution for improving diagnostic capabilities—particularly in regions with limited access to specialized medical services.

The application is designed to empower both individuals and healthcare professionals by providing **timely, automated diagnostic insights** using a deep learning model trained on the **COVID-19 Radiography Database**. Through the use of a **Convolutional Neural Network (CNN)**, the system ensures accurate and efficient classification of chest X-rays, facilitating early intervention and reducing the risk of complications.

Beyond diagnostics, the project promotes a **preventive healthcare approach**, emphasizing early screening and proactive health monitoring. By eliminating the need for expensive imaging equipment and specialist interpretation, the application helps bridge healthcare gaps, especially in **underserved and rural areas**.

Ultimately, **Chest Pneumonia Detection** seeks to demonstrate how cutting-edge **AI and web technology** can democratize access to essential healthcare tools, supporting global efforts to improve early diagnosis, health outcomes, and equity in medical services.

1.5 Scope

The **Chest Pneumonia Detection** web application is focused on delivering an efficient and accessible solution for the **early detection of pneumonia** through the automated analysis of **chest X-ray images** using deep learning.

The core scope includes:

- **Pneumonia Detection:** The system uses a **Convolutional Neural Network (CNN)** trained on the **COVID-19 Radiography Database** to classify chest X-rays as either normal or pneumonia-affected. This offers healthcare providers a rapid, AI-assisted diagnostic tool that supports clinical decision-making in real time.
- **Web-Based Accessibility:** As a web application, the system is platform-independent and easily accessible through modern browsers. It is designed to work across various devices, removing barriers to adoption and allowing broad user reach.
- **Scalable and Secure Architecture:** The application architecture supports cloud integration for secure data storage, user management, and potential future expansion. It is built with scalability in mind, allowing for the integration of additional diagnostic features and health monitoring tools.
- **Support for Underserved Regions:** By removing the need for costly diagnostic infrastructure and expert radiological services, the application addresses healthcare gaps in **remote and resource-limited environments**.

1.6 General Constraints

The development and deployment of the **Chest Pneumonia Detection** web application are influenced by several technical, legal, and operational constraints:

- **Data Privacy and Compliance:** Ensuring the protection of user-uploaded medical images is critical. The system must adhere to data privacy regulations such as **GDPR** and similar healthcare compliance standards, especially when handling potentially sensitive medical information through web interfaces.
- **Computational Resources:** Deep learning models, particularly CNNs used for image classification, are computationally intensive. While the model inference is typically handled server-side, maintaining responsiveness and cost-efficiency on cloud infrastructure is a key consideration.

- **Internet Dependency:** As a web-based solution, the application requires a stable internet connection for uploading X-ray images and receiving diagnostic results. This may limit usability in regions with poor or unreliable connectivity.
- **Dataset Limitations:** The model is trained using the **COVID-19 Radiography Database**, which, while comprehensive, may still introduce biases due to class imbalance or limited demographic diversity. Such factors can affect the model's generalizability across diverse patient populations.
- **Browser and Device Compatibility:** Ensuring the application functions correctly across a range of web browsers and devices (desktop, tablet, mobile) poses design and testing challenges.

Chapter 2:

Background and Previous work

Chapter 2 Background and Previous work

2.1 Background:

Artificial Intelligence (AI):

Artificial Intelligence (AI) refers to the simulation of human intelligence in machines, enabling them to perform tasks that typically require human cognition, such as learning, reasoning, problem-solving, perception, and language understanding. AI systems are designed to mimic human decision-making processes and can operate autonomously in complex environments [1].

Machine Learning (ML):

Machine Learning (ML) is a subset of AI that focuses on developing algorithms that allow computers to learn from data and make predictions or decisions without being explicitly programmed. Instead of relying on predefined rules, ML systems identify patterns in data and improve their performance over time through experience [2].

Types of Machine Learning Algorithms:

1. Supervised Learning:

- Definition: The algorithm learns from labeled data, where each input is paired with a corresponding output.
- Purpose: To predict outputs for new, unseen inputs based on the learned patterns.
- Common Algorithms:
 - Linear Regression: Predicts continuous values (e.g., house prices) by modeling the relationship between input variables and the output [3].
 - Logistic Regression: Used for binary classification tasks, such as spam detection [4].
 - Decision Trees: Models decisions using a tree-like structure, where each node represents a decision based on input features [5].
 - Random Forest: An ensemble method that combines multiple decision trees to improve accuracy and reduce overfitting [6].
 - Support Vector Machines (SVMs): Finds the optimal hyperplane to separate data into classes, particularly effective for high-dimensional data [7].
 - Neural Networks: Mimics the structure of the human brain to solve complex tasks, such as image recognition and natural language processing [8].

2. Unsupervised Learning:

- Definition: The algorithm learns patterns and structures from unlabeled data.
- Purpose: To explore the data or group it into clusters based on similarities.

- Common Algorithms:
 - K-Means Clustering: Divides data into clusters based on similarity, commonly used for customer segmentation [9].
 - Hierarchical Clustering: Groups data into a tree-like structure, useful for visualizing relationships between data points [10].
 - Principal Component Analysis (PCA): Reduces the dimensionality of data while preserving its variance, often used for feature extraction [11].
 - Autoencoders: Neural networks that compress and reconstruct data, useful for anomaly detection and dimensionality reduction [12].

3. Semi-Supervised Learning:

- Definition: Combines a small amount of labeled data with a large amount of unlabeled data to improve learning performance.
- Purpose: To leverage unlabeled data when labeled data is scarce.
- Example Algorithm: Self-training, where the model iteratively labels its own data and retrains itself [13].

4. Reinforcement Learning (RL):

- Definition: The algorithm learns by interacting with an environment and receiving rewards or penalties for its actions.
- Purpose: To optimize a sequence of actions to maximize cumulative rewards.
- Common Algorithms:
 - Q-Learning: Learns optimal policies for decision-making in discrete environments [14].
 - Deep Q-Networks (DQN): Combines RL with deep learning to handle complex tasks, such as playing video games [15].
 - Policy Gradient Methods: Directly optimizes the policy by maximizing expected rewards [16].

Deep Learning (DL):

Deep Learning (DL) is a subset of ML that focuses on algorithms inspired by the structure and function of the human brain, known as neural networks. These networks consist of multiple layers of interconnected neurons, enabling them to learn hierarchical representations from large datasets [17].

- **Key Features of Deep Learning:**

1. Deep Neural Networks (DNNs):

- Composed of multiple layers:
 - Input Layer: Receives raw data.
 - Hidden Layers: Extract and process features.
 - Output Layer: Produces predictions or classifications [18].

2. End-to-End Learning:

- Eliminates the need for manual feature extraction by learning features directly from data [19].

3. High Accuracy:

- Excels in tasks like image recognition, natural language processing, and speech recognition [20].

- **Common Deep Learning Architectures:**

- Convolutional Neural Networks (CNNs): Specialize in image and video processing by using convolutional layers to extract spatial features [21].
- Recurrent Neural Networks (RNNs): Suitable for sequential data, such as time series or text, by maintaining memory of previous inputs [22].
- Transformer Models: Widely used in natural language processing tasks, such as language translation and text generation [23].
- Autoencoders: Used for dimensionality reduction and anomaly detection by compressing and reconstructing data [24].
- Generative Adversarial Networks (GANs): Generate new data similar to the training data by pitting two neural networks against each other [25].

Classification:

- Definition: A type of supervised learning where the task is to predict discrete labels (e.g., spam vs. not spam, dog vs. cat).
- Models: Learn to assign inputs to predefined categories based on training data [26].

Convolutional Neural Networks (CNN):

- Definition: A specialized neural network architecture designed for image data.
- Components:
 - Convolutional Layers: Extract spatial features using filters.
 - Pooling Layers: Reduce dimensionality while preserving important features.
 - Fully Connected Layers: Combine features to produce final predictions [27].
- Applications:
 - Image classification.
 - Object detection.
 - Medical imaging analysis [28].

What is the Chest?

The chest refers to the upper part of the body, located between the neck and the abdomen. It houses critical organs, including the lungs, heart, and major blood vessels, and is protected by the ribcage, muscles, and connective tissues [29].

Pneumonia:

Pneumonia is an infection that inflames the air sacs in one or both lungs, causing them to fill with fluid or pus. Symptoms include coughing, fever, shortness of breath, and fatigue.

Pneumonia is broadly classified based on its cause [30]:

1. Bacterial Pneumonia:

- Caused by bacteria, such as *Streptococcus pneumoniae*.
- Symptoms: High fever, productive cough, and shortness of breath.
- Treatment: Antibiotics.

2. Viral Pneumonia:

- Caused by viruses, such as influenza, respiratory syncytial virus (RSV), or SARS-CoV-2 (COVID-19).
- Symptoms: Dry cough, fever, muscle aches, and fatigue.
- Treatment: Supportive care or antivirals in severe cases.

Using CNN to Classify Chest X-rays and Predict Pneumonia

CNN Classification Steps:

1. Data Collection:

- Collect a dataset of chest X-ray images labeled with categories such as:
 - Normal: No pneumonia.
 - Bacterial Pneumonia.
 - Viral Pneumonia.
- Popular datasets:
 - ChestX-ray8 [31].
 - COVID-19 Radiography Dataset [32].

2. Data Preprocessing:

- Resize Images: Normalize all X-ray images to a fixed size (e.g., 224x224 pixels).
- Normalize Pixel Values: Scale pixel values to the range [0, 1] for faster convergence.
- Augmentation: Apply transformations like rotation, flipping, and zooming to increase dataset variability.
- Split Data: Divide into training, validation, and test sets (e.g., 70-20-10) [33].

3. CNN Model Architecture:

- Use pre-trained models like VGG16, ResNet50, or EfficientNet for transfer learning [34].

4. Train the Model:

- Fine-tune the model on the pneumonia dataset using techniques like gradient descent and backpropagation [35].

5. Evaluate and Test:

- Measure performance using metrics like accuracy, precision, recall, and F1-score [36].

6. Interpret Results:

- Use visualization techniques like Grad-CAM to highlight regions of the X-ray that influenced the model's decision [37].

Advanced Considerations:

1. Use Pretrained Models:

- Leverage transfer learning with models like **VGG16**, **DenseNet201**, **InceptionV3**, **ResNet50** or **EfficientNet-B0** to reduce training time and improve performance .

2. Explainability:

- Use techniques like **Grad-CAM** to provide interpretable explanations for model predictions [38].

3. Accuracy and Reliability:

- Train on diverse and balanced datasets to ensure generalization.
- Use cross-validation to validate model robustness [39].

4. Deployment:

- Export the trained model and deploy it as a web application or API for real-time predictions [40].

2.2 Previous work:

In recent years, several studies have proposed novel approaches for automating pneumonia detection from chest X-ray images, leveraging machine learning (ML) and deep learning (DL) algorithms. These studies utilize different types of neural networks and image preprocessing techniques to improve classification accuracy and enhance the model's ability to generalize across various medical datasets. Below, we discuss the key contributions of the most relevant studies and how they differ in terms of algorithm selection, performance, and overall approach.

1. MDPI et al. (2024) [41]

The paper published in *MDPI* focuses on the application of deep convolutional neural networks (CNNs) for pneumonia detection in chest X-rays. The study explores the impact of pre-trained models such as VGG16, ResNet50, and DenseNet on the classification task. By fine-tuning these models on the pneumonia dataset, the authors achieved impressive accuracy results. The authors also emphasized the importance of hyperparameter optimization in achieving higher classification performance.

Algorithm and Results: The study primarily focuses on CNNs, using both transfer learning and fine-tuning strategies. The accuracy results for pneumonia detection were reported to exceed 90%. The main advantage of this approach is the use of pre-trained models, which significantly reduces the time and computational resources needed for training from scratch.

Difference and Relevance: This approach heavily relies on transfer learning, which may limit its ability to generalize to datasets with very different characteristics. As a result, the study emphasizes the need for further improvements in data augmentation and model tuning to address this limitation. Your work could explore additional strategies for dealing with dataset variance, such as utilizing domain adaptation techniques.

2. Alghamdi et al. (2024) [42]

Alghamdi and colleagues focus on distinguishing between bacterial and viral pneumonia using chest X-ray images, a task that requires fine-grained classification. They utilized a deep learning model built on a CNN architecture and combined it with a data augmentation technique to address the problem of class imbalance. Their findings suggest that the CNN model significantly outperforms traditional machine learning models such as support vector machines (SVM) and random forests.

Algorithm and Results: The authors employed a custom CNN architecture, which was trained using a balanced dataset. They achieved high accuracy, but their model's performance could still be improved in detecting viral pneumonia, which is often more challenging due to its subtle image features.

Difference and Relevance: Unlike the *MDPI et al.* study, which uses pre-trained models, Alghamdi et al. developed a custom CNN to tackle pneumonia classification. Your work could build on this by exploring hybrid approaches, such as integrating pre-trained models with custom layers designed to better distinguish subtle features between different types of pneumonia.

3. Cai et al. (2022)[43]

Cai and colleagues proposed a transfer learning-based model to classify pneumonia from chest X-rays. Their approach involved fine-tuning pre-trained deep neural networks, such as ResNet, on a pneumonia dataset. This method showed good performance in terms of accuracy, but it struggled when the dataset contained highly varied images.

Algorithm and Results: Transfer learning was used with pre-trained ResNet models, and the authors reported a classification accuracy of 90%. They also discussed the importance of pre-processing images to standardize the input data and improve model performance.

Difference and Relevance: This study's results are quite similar to *MDPI et al.*, but with more emphasis on the image preprocessing phase. The use of transfer learning is a key area of focus, which offers a starting point for your work. You could explore new preprocessing techniques, such as advanced image normalization or noise reduction, to further enhance the model's robustness.

4. Choi et al. (2023)[44]

Choi and collaborators focused on the deployment of pneumonia detection models in resource-constrained environments. Their approach aimed to develop lightweight models capable of operating on low-cost hardware, such as mobile devices. They achieved promising results by employing smaller, more efficient CNN architectures optimized for edge devices.

Algorithm and Results: The authors used lightweight CNN architectures, such as MobileNet, achieving reasonable accuracy (~85%) while maintaining low computational requirements. Their work is particularly valuable for clinical environments with limited resources.

Difference and Relevance: Unlike the other studies, which focus on maximizing classification accuracy, Choi et al. prioritize model efficiency. This approach might not achieve the highest possible accuracy but is ideal for real-world deployment. Your research could benefit from exploring model compression and quantization techniques to develop a balance between performance and computational efficiency.

Chapter 3:

planning and analysis

3.1 Planning

Data Availability (Extended)

Assessment: The Kaggle Chest X-Ray dataset by Paul Timothy Mooney, which contains labeled images for bacterial pneumonia, viral pneumonia, and normal cases, was selected due to its reliability and widespread use in research. Data preprocessing techniques such as resizing, normalization, and augmentation were applied to enhance model performance and account for class imbalances.

Challenges: The dataset, although publicly available, may not fully capture real-world diversity in patient demographics, imaging quality, and conditions. Additionally, noisy data and inconsistencies in labeling could introduce biases, potentially impacting model accuracy.

Resolution: Rigorous preprocessing and exploratory data analysis (EDA) were conducted to address these limitations. Future scalability plans include incorporating additional datasets for broader generalization.

Computational Resources

Assessment: Training and deploying deep learning models require substantial computational resources. High-performance GPUs were identified as essential for efficiently training convolutional neural networks (CNNs). Frameworks like TensorFlow, Keras, and PyTorch were employed for development, leveraging their robust support for DL model training.

Challenges: Resource constraints could limit model complexity, especially for ensemble techniques involving multiple architectures such as DenseNet-201, EfficientNet-B0, InceptionV3, and ResNet50.

Resolution: Google Colab and similar free cloud-based solutions were used for training. Efforts were made to optimize training processes through techniques like transfer learning and hyperparameter tuning, which significantly reduced computational overhead.

Expertise

Assessment: Successful implementation of the project required expertise in both deep learning and medical imaging. While the development team was proficient in deep learning techniques and programming frameworks, the lack of direct medical expertise posed a challenge.

Challenges: Absence of domain-specific knowledge could lead to misinterpretation of medical imaging data or failure to address nuanced clinical requirements.

Resolution: Extensive literature reviews and consultations with publicly available research helped bridge the knowledge gap. Pre-labeled datasets were relied upon to minimize the need for manual medical annotations.

Costs

Assessment: The financial aspect of the project was analyzed to ensure feasibility within the constraints of available resources. Primary costs included access to computational resources, internet bandwidth for data acquisition, and potential cloud platform expenses for deployment.

Challenges: Limited budgetary provisions made it imperative to leverage free resources.

Resolution: The project was designed with a focus on cost minimization:

- Public datasets were used, eliminating data acquisition costs.
- Open-source tools and frameworks were leveraged for development.
- Free cloud-based solutions like Google Colab were utilized for training and deployment.

System Limitations

Technical Constraints: Pre-trained models like DenseNet-201 and ResNet50, while effective, may not be fully optimized for chest disease classification without domain-specific fine-tuning. Computational resources limited the training of custom architectures or ensembles involving additional models.

Operational Constraints: The system's scope is restricted to the classification of bacterial pneumonia, viral pneumonia, and normal cases. It does not address other conditions, such as fungal infections or tuberculosis, which may also present with overlapping radiographic features.

Data Privacy: Although publicly available datasets were used, future expansion may require adherence to privacy regulations such as HIPAA and GDPR, adding complexity to implementation.

Estimated Costs

- Data Acquisition: \$0 (Public datasets were utilized).
- Development Tools: \$0 (Open-source libraries such as TensorFlow, Keras, and PyTorch were used).
- Training Resources: \$0 (Google Colab free tier was sufficient).
- Deployment Costs: \$0 (Free cloud-based deployment solutions considered).
- Miscellaneous: \$0 (Internet bandwidth was covered without additional cost).

Conclusion of Feasibility Study

The project is both technically and economically feasible within the outlined constraints. By leveraging cost-effective solutions and optimizing computational workflows, the project addresses a critical healthcare challenge. While certain limitations persist, the proposed system represents a significant step toward improving diagnostic efficiency and accuracy in resource-constrained environments. Future iterations can explore enhanced scalability, real-world integration, and expanded diagnostic capabilities.

3.1.1 Feasibility Study and Estimated Cost

Technical Feasibility

The system leverages well-established deep learning frameworks (TensorFlow, Keras, PyTorch) and pre-trained convolutional neural networks such as DenseNet-201, EfficientNet-B0, InceptionV3, and ResNet50. These technologies are widely supported and documented, ensuring technical viability. Free platforms like Google Colab were used for development and training, providing sufficient GPU resources for experimentation and model deployment.

Operational Feasibility

The workflow integrates data preprocessing, training, validation, and inference in a structured pipeline. The use of labeled, public datasets ensures reliable evaluation. The proposed system is suitable for both clinical and academic environments, and its integration can be scaled as needed depending on the healthcare facility's capacity.

Economic Feasibility

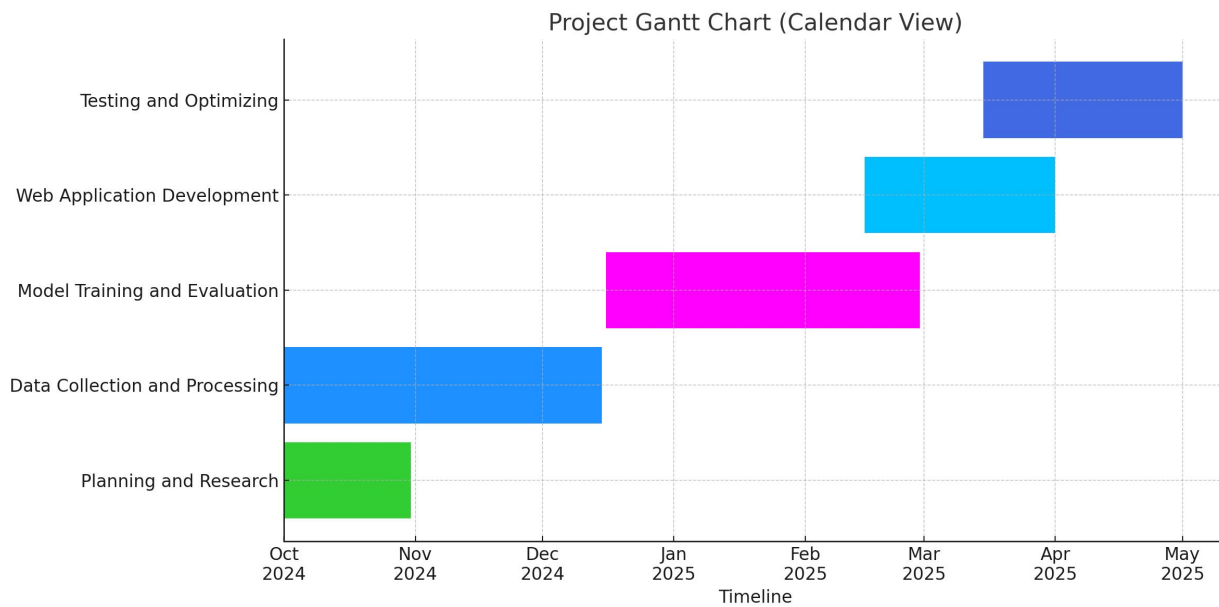
A cost-benefit analysis reveals that the project is highly cost-effective, with zero direct monetary investment due to the use of free and open-source resources.

Estimated Costs Breakdown[1]:

Component	Estimated Cost
Data Acquisition	\$0 (Public datasets used)
Development Tools	\$0 (Open-source libraries)
Training Resources	\$0 (Google Colab – free tier)
Cloud Deployment (Optional)	\$0 (Free-tier services considered)
Internet and Miscellaneous	\$0 (Existing resources used)
Total Estimated Cost	\$0

3.1.2 Gantt Chart

Below is the calendar-based Gantt chart showing the detailed project schedule across the months:



Figure[1]: Project Gantt Chart

3.2 Analysis for Chest X-Ray Pneumonia Detection:

Strengths:

Automated Diagnosis: Provides faster and consistent detection of pneumonia compared to manual interpretation.

Enhanced Accuracy: Reduces human errors by leveraging advanced AI techniques like Convolutional Neural Networks (CNNs).

Accessibility: Useful in remote areas where trained radiologists may not be available.

Scalability: Can handle large datasets and multiple requests in healthcare systems.

Early Detection: Improves patient outcomes by identifying pneumonia early, enabling timely treatment.

Applications:

- Hospitals and Clinics: Support for radiologists in detecting pneumonia cases from chest X-rays.
- Public Health: Aiding in large-scale screening during disease outbreaks (e.g., COVID-19).
- Medical Research: Assisting in the development of diagnostic tools for other lung-related diseases.

3.2 Limitations for Chest X-Ray Pneumonia Detection:

- Dataset Dependency:
 - AI models rely heavily on the quality and diversity of training data. Limited or biased datasets can lead to poor generalization.
 - Imbalanced datasets (e.g., more healthy cases than pneumonia cases) can skew predictions.
- Interpretability:
 - AI models, especially deep learning, are often considered "black boxes," making it hard for clinicians to trust the results without explainability features.
- Image Quality:
 - Poor-quality X-rays due to noise, incorrect positioning, or artifacts can reduce the system's accuracy.
- Limited Scope:
 - The system might only focus on pneumonia detection and fail to detect other abnormalities present in the X-ray.
- Regulatory Compliance:
 - Meeting medical software standards and certifications (like FDA or CE) can be a complex and lengthy process.
- Resource Requirements:
 - Training and running deep learning models require significant computational resources, which may not be feasible in resource-constrained environments.
- Ethical Concerns:
 - The system must address data privacy issues, especially when handling sensitive patient data.
 - Misdiagnoses could lead to legal and ethical consequences if not paired with proper human oversight.

3.3 Need for a New System

The accurate diagnosis of pneumonia, including its different types such as bacterial and viral, is a critical challenge in healthcare. The manual interpretation of chest X-rays can often lead to inconsistencies due to variations in expertise, workload, and time constraints faced by radiologists. In regions with limited medical resources, this challenge is even more pronounced, making timely and accurate diagnosis difficult.

A new system is needed to address these challenges by:

- **Providing Detailed Classification:** Current methods often focus on identifying whether pneumonia is present but do not distinguish between bacterial and viral types, which is crucial for appropriate treatment. A system capable of multiclass diagnosis can bridge this gap.
- **Increasing Reliability:** By reducing reliance on subjective judgment, this system ensures consistent and precise results.
- **Enhancing Accessibility:** Deploying the system in a format that can be used across different locations helps provide diagnostic support in underserved or rural areas.
- **Saving Time:** Faster analysis of chest X-rays allows healthcare professionals to prioritize critical cases and reduce the overall burden on medical staff.

3.4 Analysis of New System:

3.4.1 User Requirements:

1. **Trust and Interpretability:** Radiologists prefer systems that offer insights into how a diagnosis was made.
2. **Dataset Diversity:** Ensure the ML model is trained on a diverse dataset to handle variations in image quality, patient demographics, and disease presentations
3. The system must be able to detect diseases from X-Ray images.
4. The system can offer some advice for the patient if it detects any disease (system suggests: "Rest, hydrate, and consult a doctor immediately for antibiotics.").
5. The system can tell the patient which doctor to go to (depends on the patient region).

3.4.2 System Requirements:

To implement a system for pneumonia classification using machine learning (SVM) and deep learning (CNN), the following system requirements can be outlined:

1. Hardware Requirements

- **Processor:** Intel Core i5 or higher, or AMD Ryzen 5 or higher (preferably multi-core processors).
- **RAM:** Minimum 8 GB (16 GB or higher recommended for faster processing and training).
- **GPU:** NVIDIA GPU with CUDA support (e.g., GTX 1050 or higher, RTX 3060 or higher for larger datasets).
 - VRAM: At least 4 GB (8 GB or more is ideal for deep learning models).
- **Storage:**
 - 100 GB free disk space (for datasets, pre-trained models, and result storage).
 - SSD recommended faster data access and model training.
- **Display:** Monitor with a resolution of at least 1080p for visualization and debugging.

2. Software Requirements

- **Operating System:**
 - Windows 10/11 (64-bit)
 - Linux (Ubuntu 20.04 or later recommended for compatibility with machine learning libraries).
 - macOS (for lightweight models, but GPU support is limited compared to NVIDIA GPUs on Windows/Linux).
- **Development Environment:**
 - Python 3.7 or later.
 - Jupyter Notebook or Integrated Development Environment (IDE) like PyCharm, VS Code, or Anaconda.
- **Libraries and Frameworks:**
 - **Core Libraries:**
 - NumPy
 - Pandas
 - Matplotlib/Seaborn for visualization
 - **Machine Learning:**
 - Scikit-learn (for SVM and data preprocessing)
 - **Deep Learning:**
 - TensorFlow or PyTorch (for building and training CNN models).
 - Keras (if using TensorFlow as the backend).
 - **Data Augmentation:**
 - OpenCV or Albumentations.
- **Other Tools:**
 - CUDA Toolkit and cuDNN (for GPU acceleration with TensorFlow or PyTorch).
 - Git (for version control).
 - Kaggle API (for downloading datasets directly).

3. Dataset Requirements

- **Dataset Source:**
 - "Chest X-Ray Images (Pneumonia)" from Kaggle.
 - Ensure the dataset is pre-processed (e.g., resized, normalized, and augmented as necessary).
- **File Format:**
 - Images in JPEG or PNG format.
 - CSV/Excel files for any additional metadata (if applicable).

4. Performance Requirements

- The system should achieve a training accuracy of over 85% and a test accuracy of at least 80% for pneumonia classification tasks.
- GPU-based training should not exceed a runtime of 2 hours for a medium-sized dataset (~5,000 images).
- Memory consumption should be optimized to prevent crashes during training.

3.4.3 Functional Requirements:

1. Image Preprocessing

Input:

- Raw chest X-ray images (JPEG, PNG formats).

Preprocessing Steps:

- Resizing: Images resized to a fixed size (e.g., 224x224 or 256x256 pixels).
- Normalization: Pixel values scaled to a range of 0-1 for consistency across inputs.
- Data Augmentation: Random transformations like rotation, zoom, and flipping applied to improve model robustness.

Output:

- Preprocessed image ready for model inference.

2. Model Selection and Architecture

Model Types:

- Pre-trained models like DenseNet-201 and EfficientNet-B0 for feature extraction.
- Transfer Learning: Fine-tune models on the pneumonia dataset for improved performance on the specific task.

Output:

- A fine-tuned CNN model capable of classifying chest X-rays as either 'Pneumonia' or 'Normal.'

3. Inference and Prediction

Input:

- Preprocessed chest X-ray image.

Process:

- The model processes the image and generates a classification label ('Pneumonia' or 'Normal').
- Confidence Score: A probability score (0-1) indicating the model's confidence in the classification.

Output:

- Classification result with a confidence score, indicating the likelihood of pneumonia.

4. Frontend Requirements

Input:

- Image upload by the user (healthcare professional or patient).

Features:

- User Authentication: Login functionality with username and password for access control.
- Image Upload: Users can upload chest X-ray images (JPEG, PNG).
- Image Viewing: Display both uploaded and preprocessed images for validation purposes.
- Results Display: After image upload, the UI displays the classification result ('Pneumonia' or 'Normal') with a confidence score.
- Progress Bar: Show a progress bar indicating when the model is processing the image.
- Error Handling: Inform users of any issues with image formats or upload problems.
- Responsiveness: The UI should be responsive on both desktop and mobile devices.

Output:

- A clean, intuitive user interface displaying results, progress, and any error messages as necessary.

5. Backend Requirements

Input:

- API requests with uploaded X-ray images and patient metadata.

Process:

- Image Upload Handling: Receive and process the uploaded image, store it temporarily for model inference.
- Model Inference: Trigger the model inference process when an image is uploaded.
- Metadata Handling: Store relevant patient metadata (e.g., patient ID, image metadata) in a secure database.
- Error Handling: Ensure proper error logging and handling in case of issues like failed predictions, corrupt images, or database errors.
- Model Management: Load and invoke the pretrained model for inference based on the incoming API request.
- Response Construction: Send back the classification result and confidence score to the frontend for display.

Output:

- A backend system that handles image uploads, model inference, and response generation.

6. Database and Data Storage

Input:

- Chest X-ray images and patient metadata.

Process:

- Image Storage: Store the uploaded X-ray images temporarily or permanently, depending on system requirements (e.g., AWS S3, Azure Blob Storage).
- Metadata Storage: Store patient-related metadata (e.g., patient ID, image processing results) in a relational database (e.g., PostgreSQL, MySQL).
- Data Integrity: Ensure that images and metadata are stored correctly with proper indexing for future retrieval.

Output:

- Structured storage of images and metadata for future access or reporting.

7. User Authentication and Authorization

Frontend:

- Login/Signup: Allow healthcare professionals to create an account and log in securely.
- Session Management: Maintain user sessions for authentication and authorization.

Backend:

- JWT Authentication: Use JSON Web Tokens (JWT) to authenticate users and manage sessions.
- Role-based Access Control: Implement different access levels (e.g., admin, healthcare professional) to restrict functionality as required.

Output:

- A secure authentication system to ensure that only authorized users can upload images and view results.

8. Evaluation and Reporting

Metrics:

- Accuracy: Percentage of correctly classified images.
- Precision: Measures the ability of the model to correctly identify pneumonia cases (minimizing false positives).
- Recall: Measures the model's ability to detect actual pneumonia cases (minimizing false negatives).
- F1-Score: A balanced metric that combines precision and recall.

Output:

- Evaluation report with metrics (accuracy, precision, recall, F1-score) and a confusion matrix.

9. Compliance and Security

Frontend:

- Secure Forms: Ensure all forms (login, image upload, etc.) use HTTPS to protect data in transit.
- Data Privacy Notifications: Include clear messaging about data privacy and user consent for image processing.

Backend:

- Data Encryption: Ensure sensitive patient data is encrypted both at rest and in transit.
- Access Logs: Maintain detailed access logs for auditing purposes.
- Compliance: Ensure compliance with legal requirements (e.g., HIPAA, GDPR) for medical data.

Output:

- A compliant and secure system protecting sensitive patient information.

10. Model Deployment and Maintenance

Deployment:

- Deploy the model and backend APIs on a scalable cloud infrastructure (e.g., AWS, Azure) for real-time inference.

Monitoring:

- Continuously monitor model performance (e.g., prediction accuracy, response times) and trigger alerts if performance degrades.

Output:

- A deployable, maintainable system with real-time model inference capabilities.

3.4.4 Non-Functional Requirements:

1. Performance[2] :

Code	Requirement statement	Must/Should	Comment
NR001	The system should process and classify an X-ray image within a maximum of 5 seconds to ensure timely diagnosis.	Must	

2. Scalability[3] :

Code	Requirement statement	Must/Should	Comment
NF002	The system should handle a large number of concurrent requests (e.g., hospitals uploading multiple X-rays simultaneously). -It should be able to process datasets of varying sizes without performance degradation.	Should	

3. Usability[4] :

Code	Requirement statement	Must/Should	Comment
NF003	The interface should be simple and intuitive for medical professionals with minimal technical knowledge. -The system should support multiple languages to cater to users in different regions.	Must	

4. Availability[5]:

Code	Requirement statement	Must/Should	Comment
NF004	The application should ensure 99.9% uptime to support critical healthcare operations. -It should allow offline functionality for uploading X-rays in low- connectivity areas and sync data once online.	Should	

5. Security[6]:

Code	Requirement statement	Must/Should	Comment
NF005	All data, including X-ray images and patient information, must be encrypted during storage and transmission to comply with standards like HIPAA or GDPR.	Must	

6. Maintainability[7] :

Code	Requirement statement	Must/Should	Comment
NF006	The system should be modular to allow updates (e.g., integrating new machine learning models or enhancing features) without disrupting existing functionality. - Regular logs and monitoring tools should ensure quick identification and resolution of bugs.	Should	

7. Compliance[8] :

Code	Requirement statement	Must/Should	Comment
NF007	The system must comply with medical device regulations and AI ethics guidelines. -It should align with standards such as ISO 13485 (medical device quality management) and CE marking for medical software.	Must	

3.5 Advantages of the New System

- **Multiclass Classification Capability:** Unlike traditional binary classifiers, the new system is designed to differentiate between bacterial pneumonia, viral pneumonia, and normal chest X-rays. This allows for more targeted and effective treatment strategies, improving patient outcomes.
- **High Diagnostic Accuracy:** By leveraging deep learning models like DenseNet-201, InceptionV3, and EfficientNet-B0, the system achieves high performance in terms of sensitivity, specificity, and overall accuracy in detecting pneumonia.
- **Automated Workflow Integration:** The system can be integrated into hospital workflows, allowing for rapid pre-screening of chest X-rays before review by a radiologist, thereby enhancing clinical efficiency.
- **Cost-Effective Solution:** Utilizing publicly available datasets, open-source tools, and free cloud-based resources such as Google Collab ensures minimal financial burden, making the system viable for low-resource settings.
- **Scalability and Portability:** The system can be deployed across a wide range of platforms—from local servers to cloud applications, making it adaptable for small clinics as well as large healthcare institutions.
- **Decision Support:** Acts as a second opinion for clinicians, helping to validate or question initial diagnoses, thus reducing the risk of human error.
- **Reduced Radiologist Workload:** Automating the initial diagnostic phase allows radiologists to focus on more complex cases and clinical decisions, ultimately improving service quality and reducing burnout.
- **Improved Patient Triage:** Enables faster identification of critical pneumonia cases, allowing healthcare professionals to prioritize patient care more effectively.

3.6 User Characteristics

The system is designed for the following types of users:

1. Doctors

- **Roles and Permissions:**

- Upload X-ray images to the system for diagnosis.
- Monitor patient conditions, prepare medical reports, and share findings with patients.
- Prioritize urgent cases based on system results.

- **User Requirements/9/:**

Requirement	Details
Age	Above 28 years.
Gender	Both genders.
Education Level	Master's degree in medicine or radiology.
Previous Experience	Preferably experienced in reading X-rays.
Special Skills	Basic knowledge of using desktop applications.

2. Patients

- **Roles and Permissions:**

- Send symptoms to the system for analysis.
- Receive recommendations for specialists based on their condition.
- Book appointments with doctors or request detailed health reports.

- **User Requirements[10]:**

Requirement	Details
Age	All ages.
Gender	Both genders.
Education Level	Not required.
Previous Experience	No prior experience required.
Special Skills	Basic familiarity with smartphones or computers.

3. Researchers[11]

- **Roles and Permissions:**

- Analyze data and study disease patterns using the system.
- Classify cases and test new diagnostic methods.

- **User Requirements:**

Requirement	Details
Age	Above 25 years.
Gender	Both genders.
Education Level	Preferably a master's or Ph.D. in medical or research fields.
Previous Experience	Experience in research and medical data analysis.
Special Skills	Familiarity with medical analysis tools and software.

4. Medical Students/12/

- **Roles and Permissions:**

- Use the system as a learning tool to practice interpreting X-ray images.
- Access sample cases and corresponding diagnoses.

- **User Requirements:**

Requirement	Details
Age	Above 18 years.
Gender	Both genders.
Education Level	Enrolled in a medical or radiology program.
Previous Experience	No prior experience required.
Special Skills	Basic knowledge of educational tools and applications.

5. Medical Technicians *[13]*

- **Roles and Permissions:**

- Prepare and process diagnostic images for analysis.
- Assist doctors in managing cases that require urgent review.

- **User Requirements:**

Requirement	Details
Age	Above 22 years.
Gender	Both genders.
Education Level	Diploma or technical certificate in radiology or medical fields.
Previous Experience	Experience in handling medical equipment and imaging software.
Special Skills	Ability to operate medical systems and devices.

- **Common User Needs**

1. **Clarity:**

- Clear and straightforward presentation of results, including visual indicators on X-rays to highlight affected areas.

2. **Efficiency:**

- Fast image processing provides timely results and support quick decision-making.

3. **Reliability:**

- Consistent and accurate classification of chest X-rays into normal, bacterial pneumonia, or viral pneumonia categories.

4. **User-Friendly Design:**

- A system interface designed to accommodate users with varying levels of technical expertise, ensuring ease of use for all.

Chapter 4: design

Chapter 4: design

4.1 Design and Implementation Constraints:

4.1.1 Technical Constraints

- **Server Performance and Load Handling:** As a web-based application, all AI model inferences occur on the server side. Efficient load balancing and optimized model deployment (e.g., using TensorFlow Serving or ONNX) are essential to ensure the application remains responsive during high user traffic.
- **AI Model Limitations:** The reliability of the pneumonia detection model is dependent on the COVID-19 Radiography Database used for training. Limitations in dataset diversity (e.g., age, ethnicity, imaging quality) can affect generalizability, necessitating ongoing dataset expansion and validation.
- **Internet Dependency:** Since users must upload chest X-ray images for processing, a stable internet connection is essential. This can limit accessibility in areas with poor connectivity. Offline use is not feasible for model inference, though lightweight local versions could be explored for future development.
- **Browser and Device Compatibility:** The application must maintain consistent functionality across all major web browsers (Chrome, Firefox, Edge, Safari) and devices (desktop, tablet, mobile). UI responsiveness and image upload functionality must be tested and optimized for various screen sizes and performance profiles.
- **Cloud Infrastructure and Cost Management:** Hosting a CNN-based model on cloud servers (e.g., Google Cloud) requires managing both costs and resource scalability. Efficient containerization (e.g., Docker) and auto-scaling strategies are needed to balance performance with affordability.

4.1.2 Regulatory and Legal Constraints

- **Data Privacy and Security:** As the web application processes sensitive medical images, strict compliance with data protection regulations such as GDPR and relevant healthcare data standards is essential. All user data—including uploaded chest X-rays—must be encrypted both in transit and at rest. Measures such as data anonymization, secure file handling, and access control are necessary to safeguard patient privacy.
- **Ethical AI Usage:** The deployed CNN model must align with principles of ethical AI in healthcare, including transparency, fairness, and accountability. Misdiagnosis or bias in predictions can lead to serious consequences; thus, regular model evaluation, retraining, and explainability mechanisms (e.g., Grad-CAM or saliency maps) should be employed to interpret AI decisions and ensure responsible usage.

4.1.3 Resource Constraints

- **Development Resources:** The project requires collaboration among skilled AI engineers, web developers, UI/UX designers, and cloud infrastructure specialists. Limited access to such resources can affect implementation speed and quality. Agile project management and phased development are needed to ensure efficient resource utilization.

- **Budget Limitations:** Operating within a limited budget demands strategic use of open-source tools and cloud credits (e.g., AWS Free Tier, Google Colab for prototyping) for model training and deployment. Cost-effective services must be prioritized for data storage, backend APIs, and web hosting without compromising performance or security.
- **Time Constraints:** With a development window of six months, the project must adhere to a tightly managed timeline covering planning, prototyping, testing, and deployment. A detailed Gantt chart, milestone tracking, and continuous integration workflows will be essential to avoid bottlenecks and ensure timely delivery.

4.2 Assumptions and Dependencies

4.2.1 Assumptions

- **User Access to Internet and Devices:** It is assumed that users will have access to a device with a modern web browser (desktop, tablet, or smartphone) and a stable internet connection. This enables users to upload chest X-ray images and receive diagnostic predictions via the web interface.
- **X-ray Image Quality and Format:** The accuracy of AI-based classification assumes that users will upload valid and properly formatted chest X-ray images (e.g., JPEG or PNG). Clear instructions and validation checks will be integrated into the upload interface to reduce input errors.
- **Cloud Service Reliability:** The application assumes reliable performance from cloud infrastructure providers (e.g., Firebase, AWS, or Google Cloud) for secure data storage, backend APIs, and AI model inference. Redundant systems are planned to mitigate the risk of service outages.
- **AI Model Maintenance:** The design assumes that AI models will be regularly retrained and updated with newer, high-quality datasets like the COVID-19 Radiography Database. The system's modular architecture supports seamless integration of updated models without interrupting live service.

4.2.2 Dependencies

- **Cloud Infrastructure Providers:** The system depends on third-party cloud services for backend operations, data storage, and AI model hosting. Tools like Firebase are used for authentication and real-time data handling. Alternate solutions (e.g., AWS S3, Google Cloud Storage) are considered for redundancy and flexibility.
- **Medical Image Datasets:** The pneumonia detection model is trained using the COVID-19 Radiography Database hosted on Kaggle. Continuous access to updated, medically validated datasets is essential to maintain model accuracy and relevance.
- **Web Hosting and Deployment Services:** The application relies on secure, scalable web hosting platforms to ensure uptime, global access, and HTTPS support. This includes platforms like Vercel, Netlify, or custom cloud VM deployments.
- **Medical Expertise and Review:** Collaboration with radiologists or healthcare professionals is necessary to validate the AI model's outputs and improve clinical

reliability. This dependency is critical to maintaining ethical standards and real-world applicability.

- **Regulatory Compliance:** Deployment of the application in different regions depends on compliance with local and international data protection and medical device regulations. This includes meeting the requirements set by health data laws such as GDPR and regional approval bodies.

4.3 Risks and Risk Management

4.3.1 Technical Risks

- **AI Misclassification of Pneumonia:** The CNN model used for chest X-ray classification may produce incorrect predictions due to limitations in training data quality or distribution (e.g., low representation of specific age groups or imaging conditions).
 - **Mitigation:** Incorporate expert-reviewed datasets such as the **COVID-19 Radiography Database** and involve medical professionals in validating results. Include clear disclaimers that predictions are preliminary and not a replacement for clinical diagnosis. Enable regular retraining with new, diverse, and validated data.
- **Server Downtime and Latency:** As a web-based system, the application is susceptible to server outages or performance bottlenecks, especially under high traffic conditions.
 - **Mitigation:** Use scalable, containerized infrastructure (e.g., Docker, Kubernetes) with load balancing and auto-scaling on cloud platforms. Conduct regular performance testing and monitor uptime with health checks and alerts.
- **Browser and Device Compatibility Issues:** Users accessing the application via various browsers and devices may encounter inconsistent behavior, especially during image uploads or rendering results.
 - **Mitigation:** Apply responsive web design principles and conduct cross-browser testing. Use JavaScript-based image validation tools to standardize preprocessing across devices.

4.3.2 Security and Privacy Risks

- **Sensitive Data Exposure:** The application handles chest X-ray images, which may contain identifiable health information. There is a risk of data leaks or unauthorized access.
 - **Mitigation:** Enforce HTTPS, encrypt files at rest and in transit, and implement role-based access controls. Use **end-to-end encryption** and secure user authentication protocols (OAuth 2.0, MFA). Regularly audit infrastructure for vulnerabilities.
- **Non-Compliance with Data Regulations:** Deployment in different regions may involve legal exposure if data privacy regulations like **GDPR** are not strictly followed.
 - **Mitigation:** Establish a data governance policy aligned with GDPR and similar frameworks. Consult legal experts to implement consent mechanisms and data deletion rights.

4.3.3 Operational Risks

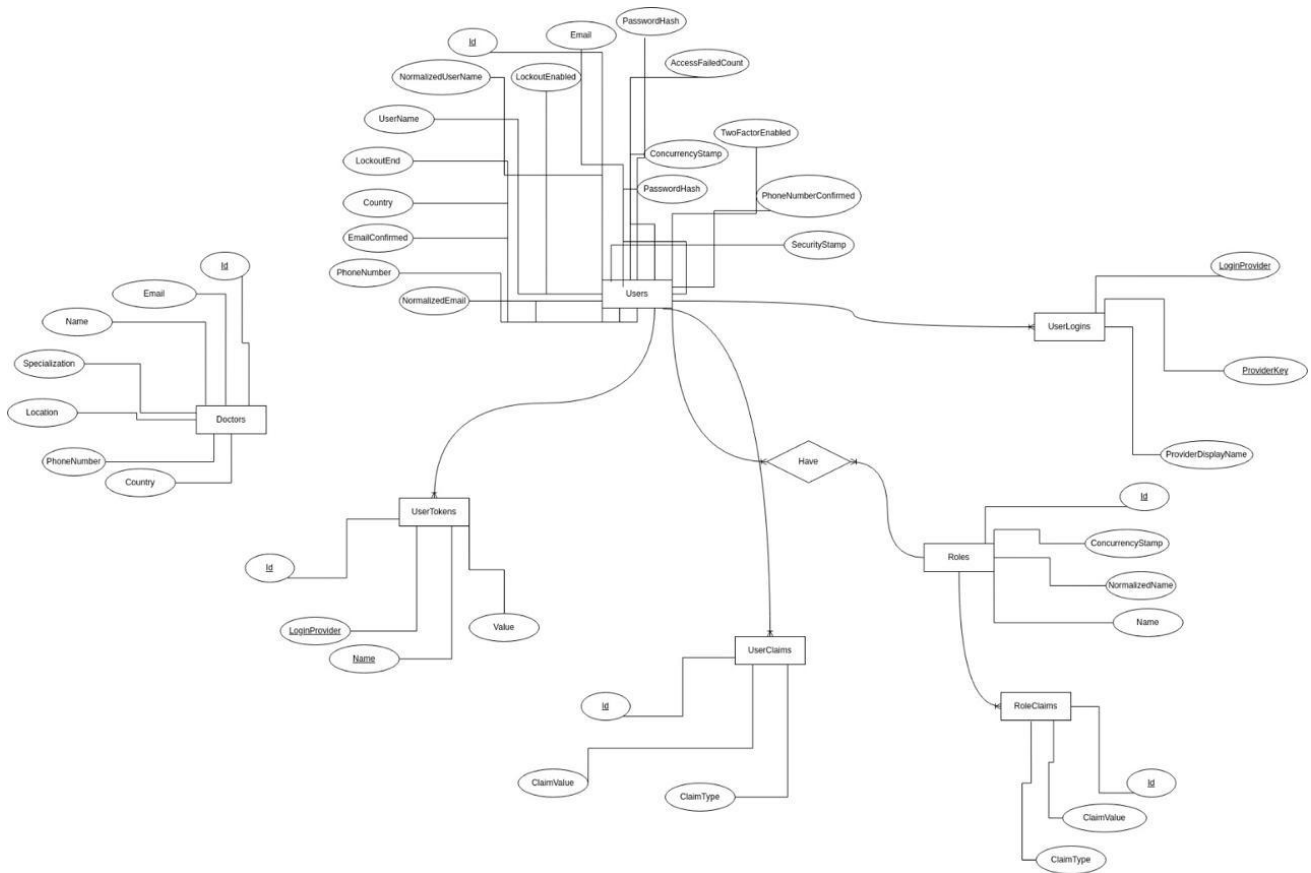
- **User Hesitation or Mistrust:** Users may distrust the accuracy or safety of an AI-based diagnostic tool, leading to low adoption rates.
 - **Mitigation:** Build transparency into the system by offering explainable outputs (e.g., visual heatmaps showing pneumonia-detected regions). Include educational material and disclaimers, and partner with health professionals or institutions to build credibility.
- **Limited Technical and Financial Resources:** Constraints in budget, infrastructure, or skilled personnel could delay development and maintenance.
 - **Mitigation:** Leverage open-source frameworks (e.g., TensorFlow, React, Flask), apply for cloud credits (AWS/GCP), and prioritize development in phases. Seek academic or institutional collaborations for technical support.

4.3.4 Ethical Risks

- **Bias in AI Predictions:** CNN models may underperform for specific demographic groups if the dataset lacks diversity, leading to unequal diagnostic accuracy.
 - **Mitigation:** Use demographically diverse training data and implement fairness-aware evaluation metrics. Involve clinical advisors in model auditing to detect and correct bias early.
- **Over-Reliance on AI Recommendations:** Users may mistakenly treat AI results as final medical conclusions, delaying professional care in critical cases.
 - **Mitigation:** Prominently display disclaimers and instructions encouraging users to consult a physician. Label AI predictions clearly as “preliminary” and avoid definitive diagnostic language.

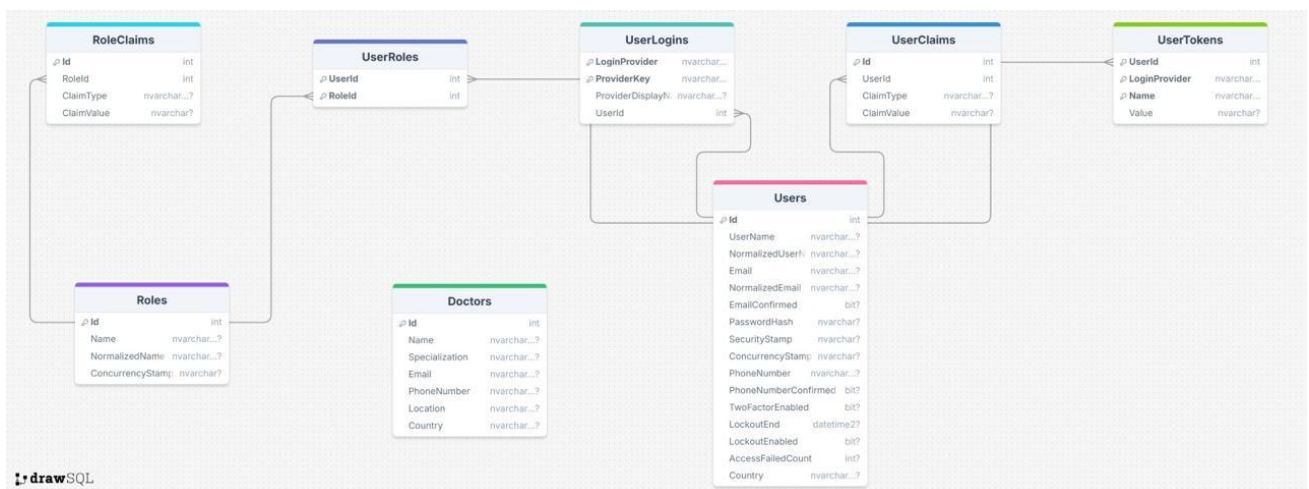
4.4 Design of database ERD

4.4.1 Entity Relationship Diagram



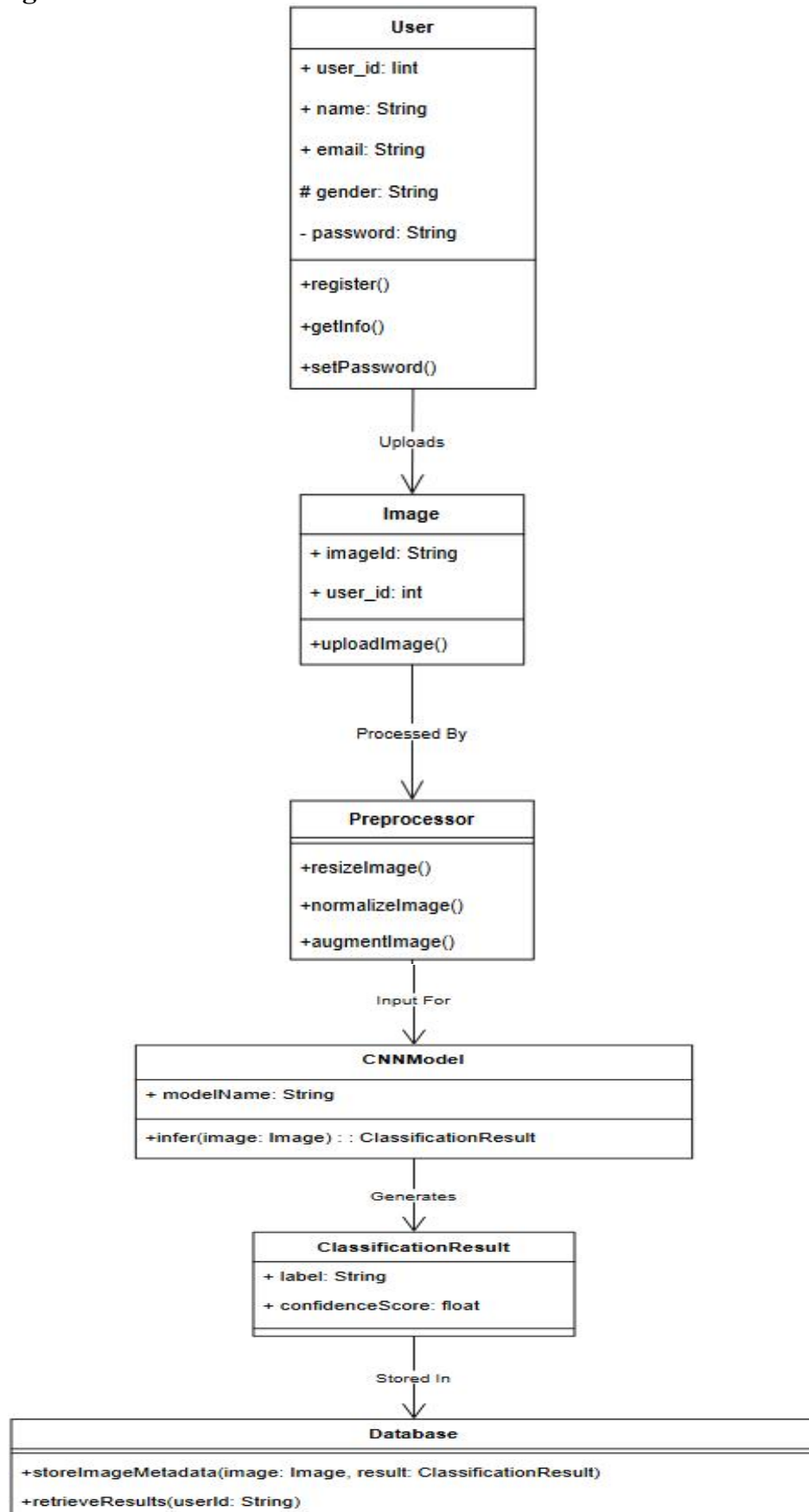
Figure[2]: Entity relationship Diagram

4.4.2 Mapping of Entity Relationship Diagram



Figure[3]: Mapping of Entity Relationship

4.4.3 Class Diagram



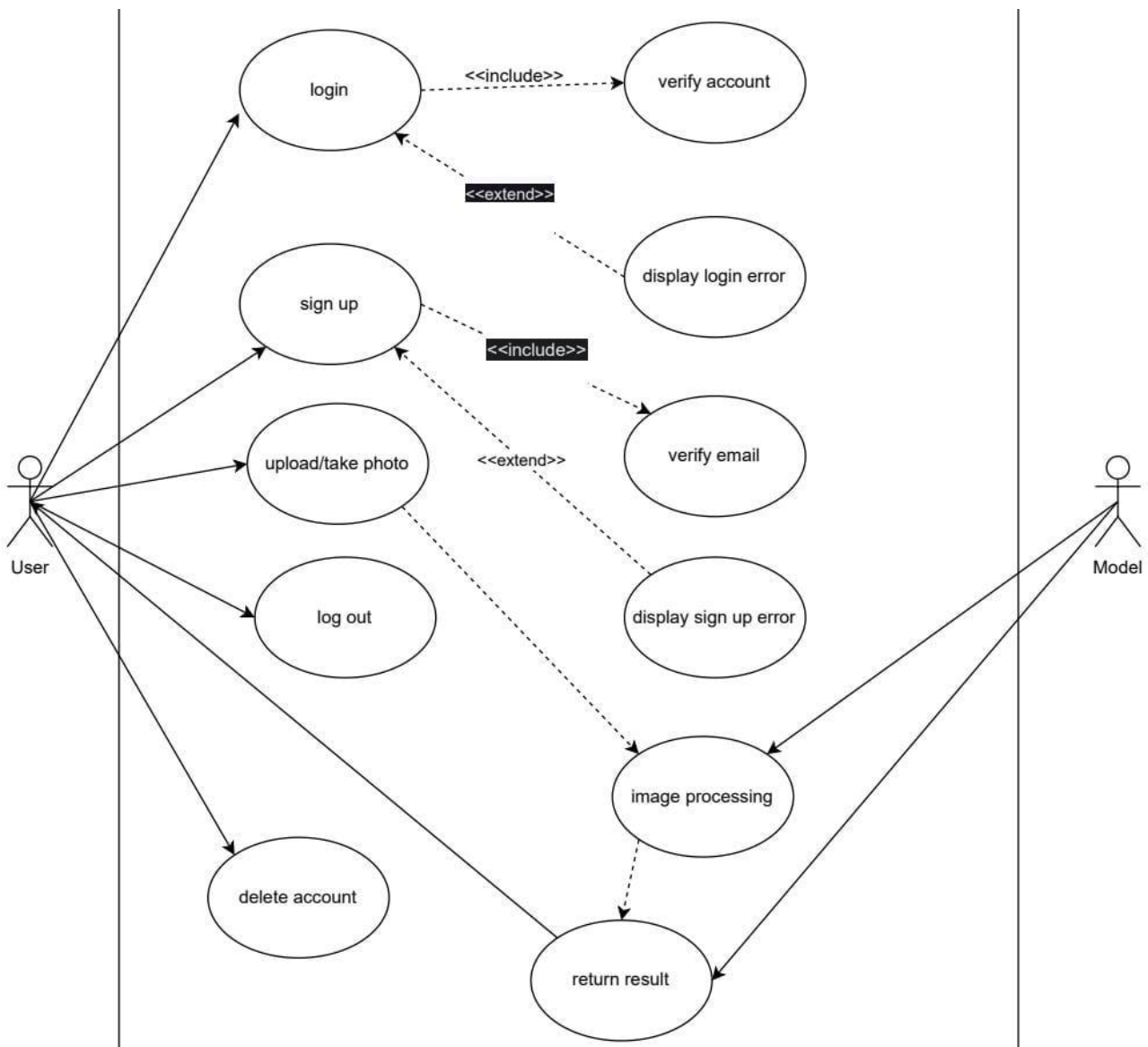
Figure[4]: Class Diagram

4.5 Use case diagram

4.5.1 Primary Use-case Diagram:

A use case diagram illustrates the primary interactions between users and the Chest Pneumonia Detection Web Application. It visually represents how users engage with various system functionalities to achieve specific goals within the application.

In this context, the actor is the Application User, who interacts with the system primarily to upload chest X-ray images, receive AI-based pneumonia predictions, view historical results, and access educational information.



Figure[5]: Use-case Diagram

4.5.2 Use-case scenario

Use Case Title:

Detect Pneumonia from Chest X-Ray

Primary Actor:

Application User (e.g., patient, general user, or healthcare staff)

Stakeholders and Interests:

User: Wants fast, accurate results for pneumonia detection based on an uploaded chest X-ray

System Owner/Developer: Wants to provide an intuitive, reliable, and medically compliant tool.

Healthcare Professionals (optional): May use the tool to support early screening.

Preconditions:

- The user has access to a device with an internet connection and a browser.
- The user has a digital chest X-ray image (in JPG/PNG format).
- The system is online and the AI model is loaded.

Main Success Scenario (Basic Flow):

1. User accesses the web application through a browser.
2. System displays the home page, providing instructions for uploading a chest X-ray image.
3. User selects and uploads a chest X-ray image via the upload interface.
4. System validates the image format and quality.
5. System sends the image to the AI model, which runs inference using a pre-trained CNN.
6. System receives prediction results (e.g., “Pneumonia Detected” or “No Pneumonia Detected”) with a confidence score.
7. System displays the result to the user along with a disclaimer that this is not a medical diagnosis.
8. User optionally views additional resources such as explanations, health tips, or uploads another image.
9. User ends the session or logs out (if login is implemented).

Alternative Flows:

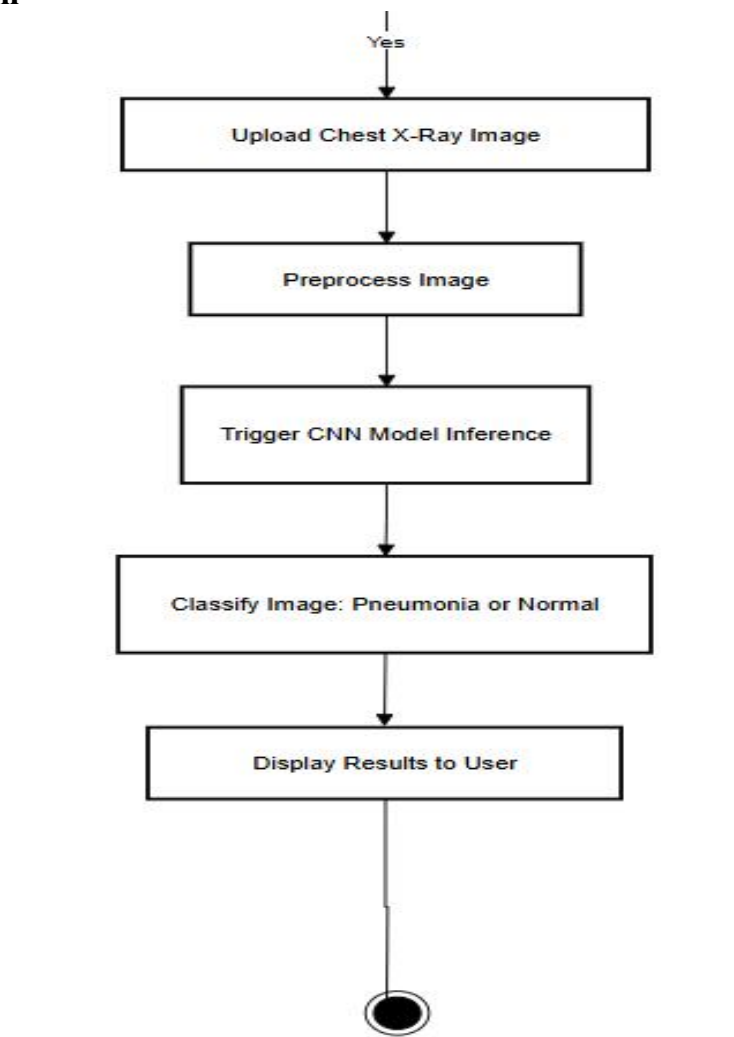
- **A1: Invalid Image Format**
 - System detects unsupported file type.
 - Displays error message and asks the user to re-upload a valid image (e.g., .jpg, .png).
- **A2: Server Error or Timeout**
 - If inference service is unavailable or times out, the system notifies the user and suggests retrying later.
- **A3: User Is Not Logged In (if applicable)**

- The system restricts access to certain features like viewing previous results or saving images unless logged in.

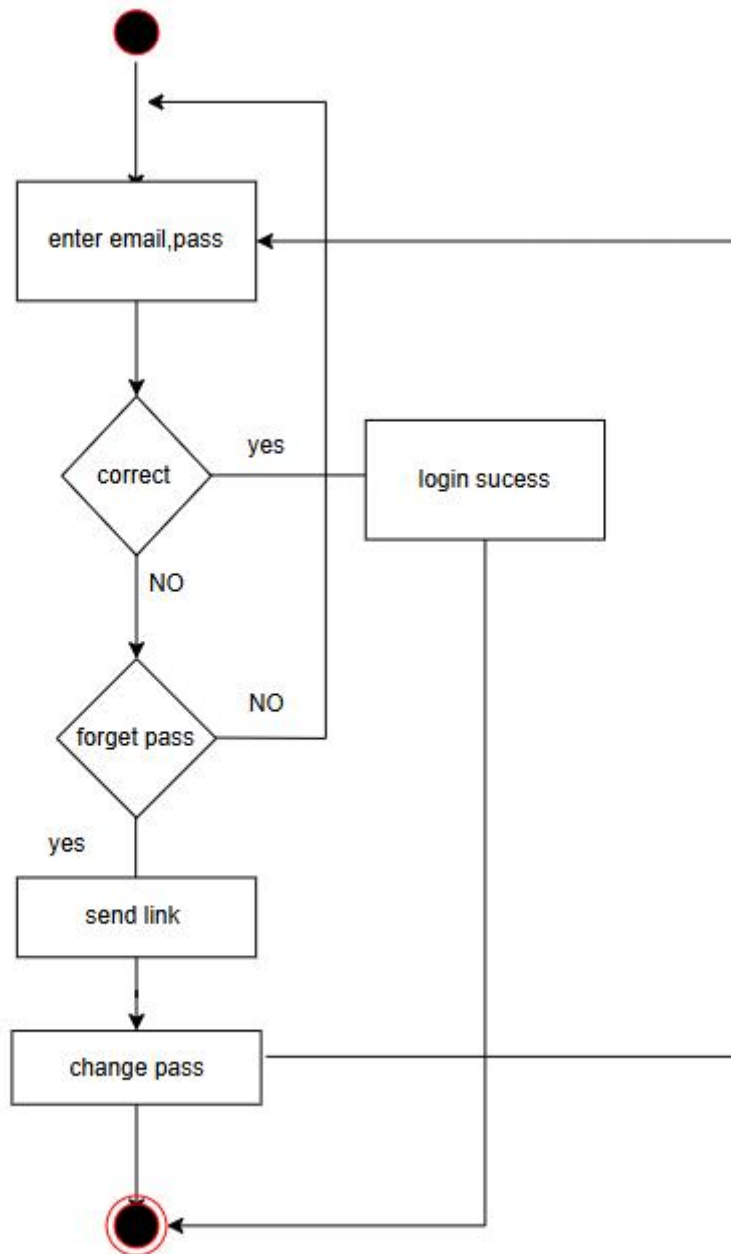
Postconditions:

- The user receives an AI-generated prediction regarding the presence or absence of pneumonia in the X-ray.
- The image and result may be saved in the system if the user is authenticated and storage is enabled.

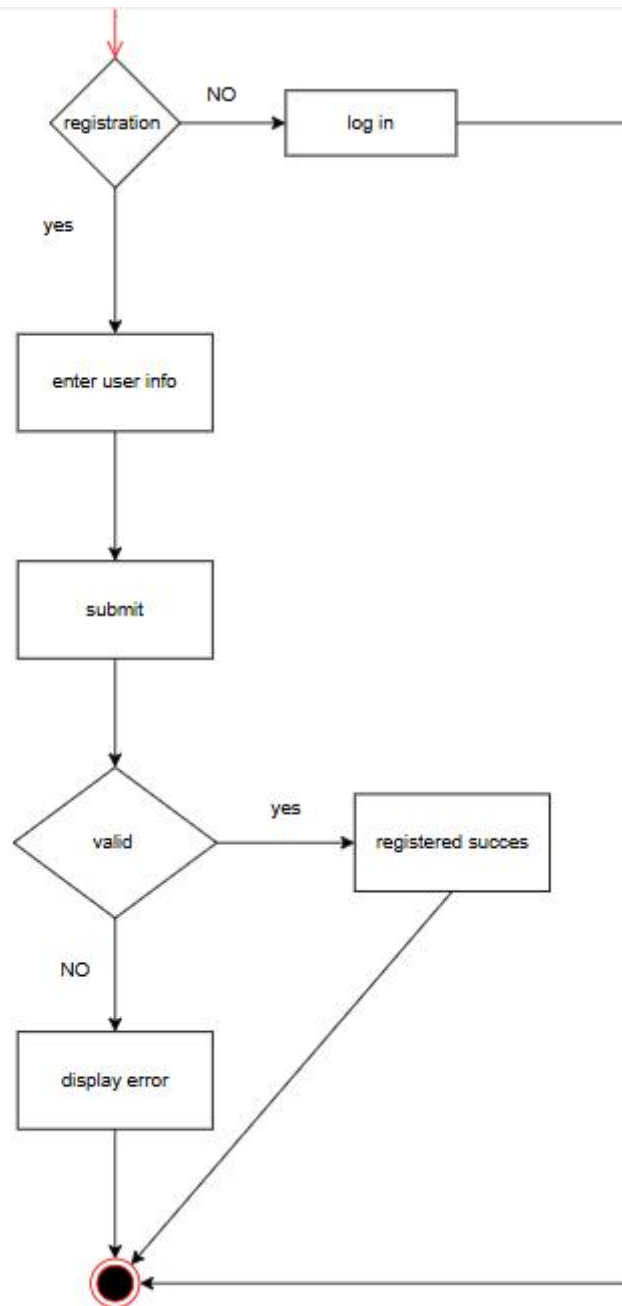
4.6 Activity Diagram



Figure[6]: Activity Diagram

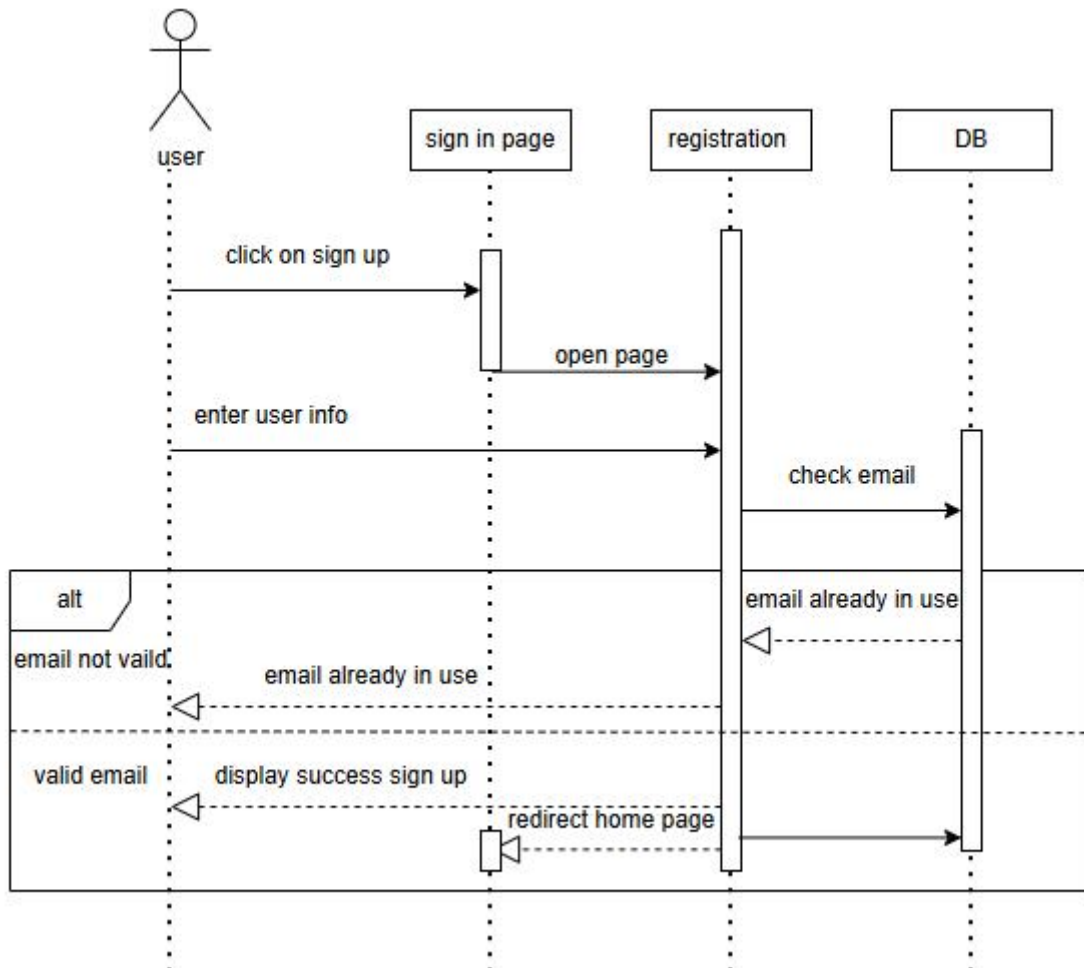


Figure[7]: Activity Diagram



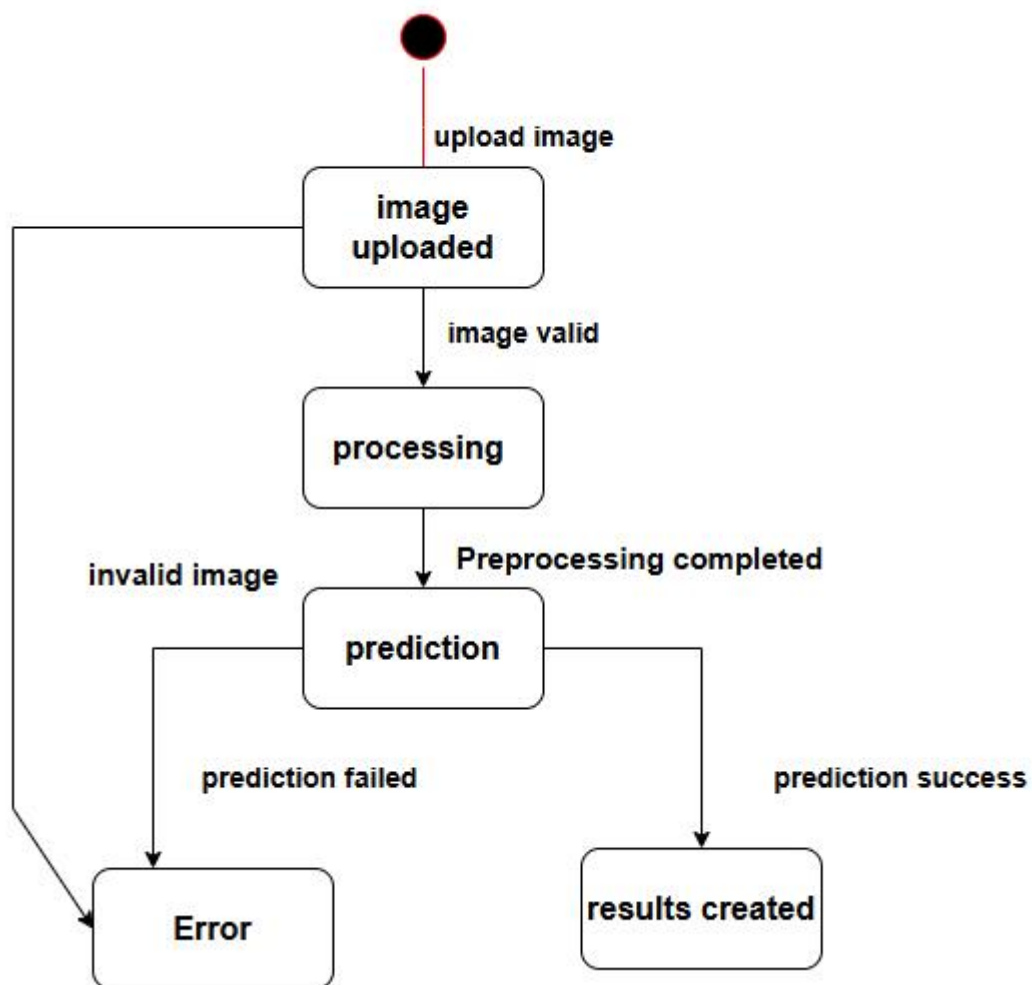
Figure[8]: Activity Diagram

4.7 Sequence Diagram



Figure[9]: Sequence Diagram

4.8 State Diagram



Figure[10]: State Diagram

Chapter4: Implementation

5.1 Software Architecture

1. Chest Pneumonia Classification Model

We began our work on this problem with the recognition that obtaining large volumes of well-labeled chest X-ray images for pneumonia detection is both important and challenging. To achieve high accuracy and generalization in medical image classification tasks, we employed **transfer learning**, which has proven effective in leveraging pre-trained models on large-scale datasets.

This method enables the development of robust classification models even when domain-specific labeled data is limited. We applied transfer learning on a substantial dataset of chest radiographs and used **data augmentation techniques** to improve generalization and reduce overfitting, thus making the model more adaptable to real-world data.

To enhance our model's performance, we utilized the **COVID-19 Radiography Database** from Kaggle, which includes a total of over 21,000 chest X-ray images categorized into four classes:

- COVID-19
- Viral Pneumonia
- Lung-Opacity
- Normal

By applying augmentation techniques such as rotation, flipping, zooming, and contrast adjustments, we artificially expanded our dataset. This increased the number and variability of training samples, resulting in a more balanced and diverse dataset. As a result, we significantly improved the model's ability to detect and classify different types of pneumonia, especially in ambiguous or overlapping cases.

This enriched dataset helped in training a deep convolutional neural network (CNN) capable of accurately classifying chest X-ray images across multiple pneumonia categories.

Ensemble Model Architecture

This system implements a deep learning-based pipeline for automated multi-class classification of chest X-ray images. The architecture is designed to handle complex clinical data with a high degree of variability, leveraging an ensemble of fine-tuned convolutional neural networks to improve diagnostic accuracy.

The pipeline begins with input chest X-ray images, typically in PNG or JPG formats, standardized to a resolution of 224x224 RGB channels. A comprehensive preprocessing stage is applied, including resizing, cropping, normalization, and extensive data augmentation techniques such as random cropping, flipping, rotation, color jitter, perspective transformation, and affine transformations. These steps enhance the model's robustness to variations in real-world clinical imaging data.

1. Data Preparation

1.1 Input Data

Image Format: PNG, JPG

Color Channels: RGB

Input Resolution: 224x224 pixels

1.2 Preprocessing

Resizing

Center Crop

Normalization (mean/std based on ImageNet pretrained models)

1.3 Data Augmentation

Random Crop

Random Horizontal Flip

Random Rotation

Color Jitter

Perspective Transformation

Affine Transformations

1.4 Dataset Organization

Folder structure compatible with `torchvision.datasets.ImageFolder`

Class folders representing each label

1.5 Dataset Split

1.6 Data Loading

Subset	Percentage
Train	70%
Validation	15%
Test	15%

DataLoaders: train_loader, val_loader, test_loader

Batch Size: 64

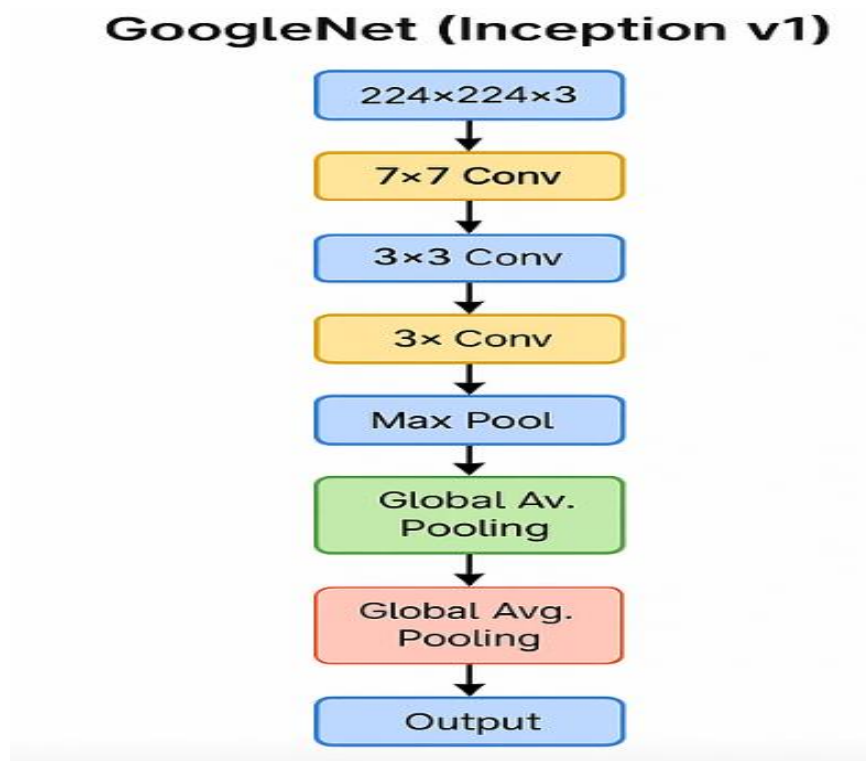
Parallelization: num_workers=4, pin_memory=True

2. Model Architecture

We have 3 individual pretrained models which process the same input image in parallel:

Google Net (Inception v1) 2.1

Architecture Summary:



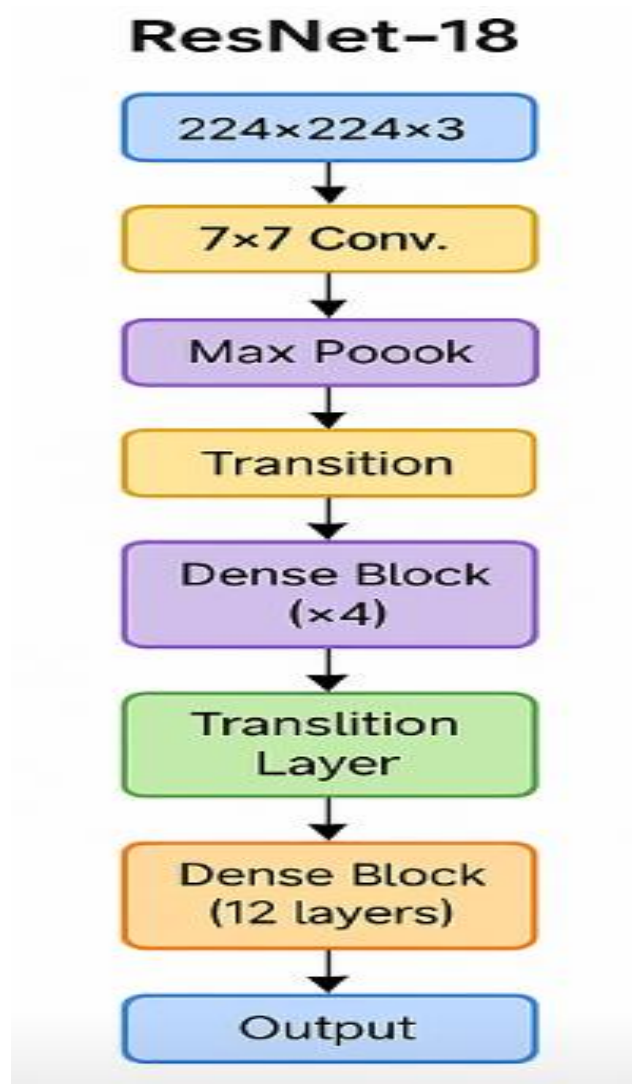
Figure[11]: GoogleNet Architecture

Key Feature:

- Uses Inception modules: multi-scale convolutions (1x1, 3x3, 5x5) + pooling inside one block.
- Feature extraction is highly diversified.

Output:

- Logits of shape (batch_size, 4).
-

2.2 ResNet-18**Architecture Summary:**

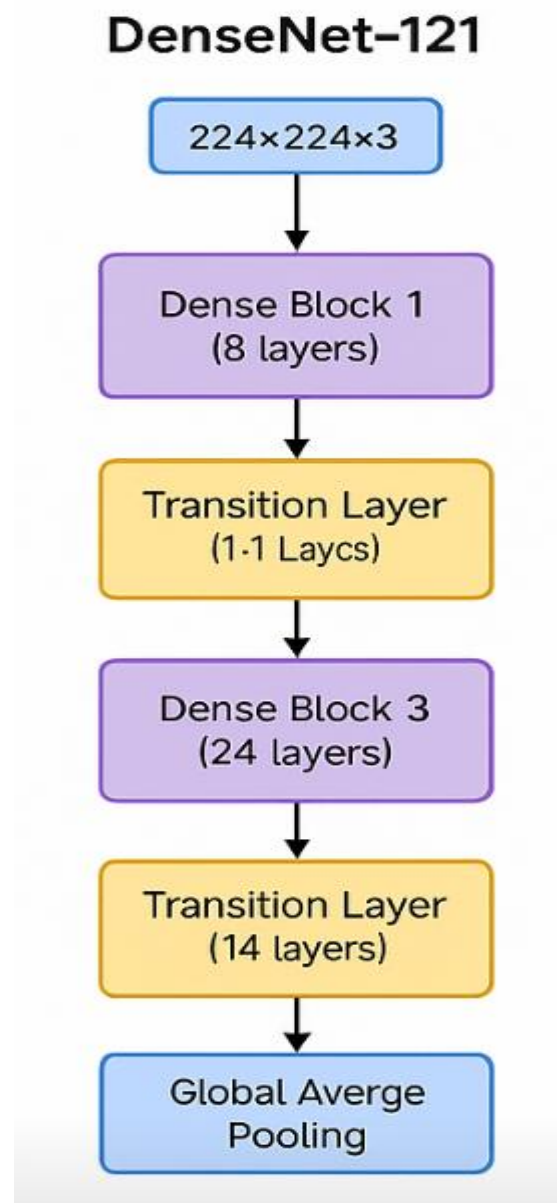
Figure[12]: ResNet-18 Architecture

Key Feature:

- Skip connections (shortcut paths) to prevent vanishing gradients.
- Simpler architecture but strong performance.

Output:

- Logits of shape (batch_size, 4).

2.2 DenseNet-121**Architecture Summary:**

Figure[13]: DenseNet-121 Architecture

Key Feature:

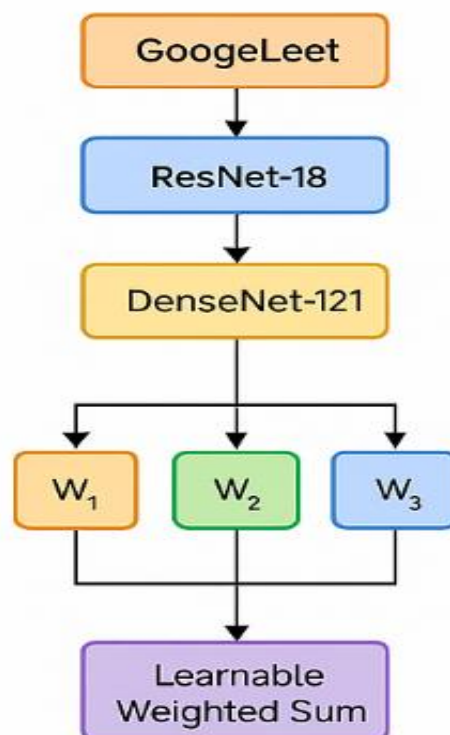
- Dense connections: each layer receives input from all previous layers.
- Strong feature reuse, reduced parameter count.

Output:

- Logits of shape (batch_size, 4).

3. Ensemble Layer:

Ensemble Model



Figure[14]: Ensemble Model Architecture

Step 1:

Each model generates independent outputs:

- GoogLeNet output: G
- ResNet output: R
- DenseNet output: D

Each output shape: (batch_size, 4)

Step 2:

Introduce learnable weights:

- $W = [W_1, W_2, W_3]$
- Initialize $W = [1.0, 1.0, 1.0]$ (trainable parameter)

Step 3:

Apply Softmax to normalize weights:

$$W_{\text{normalized}} = \text{Softmax}(W) \quad W_{\text{normalized}} = \frac{\text{Softmax}(W)}{\sum \text{Softmax}(W)}$$

- Ensures $W_1 + W_2 + W_3 = 1$

Step 4:

Weighted Sum:

$$\text{Output} = W_1 \cdot G + W_2 \cdot R + W_3 \cdot D \quad \text{Output} = W_1 \cdot G + W_2 \cdot R + W_3 \cdot D$$

$$D_{\text{Output}} = W_1 \cdot G + W_2 \cdot R + W_3 \cdot D$$

- Combine logits from all models.
- Still shape (batch_size, 4).

Step 5:

Final Prediction:

- Apply $\text{Softmax}()$ over output logits for final class prediction.

4. Loss Layer

Loss Function: Focal Loss

The system employs **Focal Loss** to address class imbalance, a common issue in medical image classification tasks where certain disease classes may be underrepresented. Focal Loss modifies the standard cross-entropy loss by down-weighting easy-to-classify examples and focusing the model's learning on harder, misclassified samples.

- **Purpose:**
 - Handle class imbalance.
 - Focus on difficult, minority, or borderline samples.
- **Focal Loss Formula:**

$$FL = \alpha(1 - p_t)^\gamma \cdot CE$$

Where:

- “ p_t ” is the model’s estimated probability for the true class.
- “ α ” is the balancing factor (usually between 0 and 1).

- “ γ ” is the focusing parameter (≥ 0) that adjusts the rate at which easy examples are down-weighted.
- “CE” is the standard cross-entropy loss.
- **Typical Hyperparameters Used:**
 - $\alpha=0.25$
 - $\gamma=2.0$

5. Optimization Layer

Optimizer: AdamW

- **Description:**
AdamW is used for optimization. It is an extension of the Adam optimizer that decouples weight decay from the gradient update, which can lead to better generalization.
- **Key Benefits:**
 - Better weight decay handling.
 - Suitable for fine-tuning pretrained models.

Learning Rate Scheduler: ReduceLROnPlateau

- **Scheduler Function:**
Reduces the learning rate when a monitored validation metric has stopped improving.
- **Configuration:**
 - Monitor: Validation Loss
 - Patience: (e.g. 3 epochs)
 - Factor: Learning rate is reduced by a constant factor (e.g. 0.1)

Early Stopping

- **Description:**
Training is terminated early if no improvement is seen in the validation loss after a fixed number of consecutive epochs.
- **Configuration:**
 - Patience: 5 epochs

6. Evaluation Layer

- **Metrics:**
 - Accuracy:

Final Test Results - Loss: 0.2020, Accuracy: 85.27%

- Precision, Recall, F1-Score
- Confusion Matrix
- **Visualization:**
 - Training & Validation Loss Curves
 - Accuracy Curves
 - Classification Report
 - Confusion Matrix Heatmap

Conclusion

In the initial phase of this project, an ensemble-based deep learning model was developed for multi-class classification of chest X-ray images. This ensemble combined multiple pretrained convolutional neural networks (GoogLeNet, ResNet-18, DenseNet-121) with a learnable weighted fusion layer. Despite employing advanced preprocessing techniques, data augmentation, focal loss to handle class imbalance, and optimization strategies (AdamW optimizer, ReduceLROnPlateau scheduler, and early stopping), the ensemble model did not fully meet the desired level of classification performance.

Based on these findings, a fully customized convolutional neural network (CNN) model was subsequently designed and implemented, tailored specifically for chest X-ray image characteristics. The custom CNN architecture allowed for more efficient feature extraction, better generalization, and improved diagnostic accuracy across all target classes. As a result, the CNN-based model significantly outperformed the initial ensemble model, demonstrating stronger robustness and suitability for potential clinical deployment.

CNN Architecture

1. Input Layer

- **Purpose:** Takes the raw image data as input.
- **Format:** Usually a 3D array of shape (height, width, channels).

2. Convolutional Layer (Conv Layer)

- **Purpose:** Detects patterns such as edges, textures, and shapes by applying filters (kernels).
- **How it works:**
 - Applies small filters (e.g., 3x3 or 5x5) that slide across the input.
 - Computes **dot products** between the filter and input region to produce a **feature map**.
- **Key Concepts:**
 - **Stride:** How much the filter moves.

- **Padding:** Adds zeros around the image to maintain size.

3. Activation Layer (ReLU)

- **Purpose:** Introduces non-linearity to the model.
- **Common activation:** ReLU (Rectified Linear Unit): $f(x) = \max(0, x)$
- **Why it's needed:** Without non-linear activation, CNN would behave like a linear model.

4. Pooling Layer (Downsampling)

- **Purpose:** Reduces the spatial size (height and width), making computation faster and reducing overfitting.
- **Types:**
 - **Max Pooling:** Takes the max value in a region.
 - **Average Pooling:** Takes the average value.

5. Flatten Layer

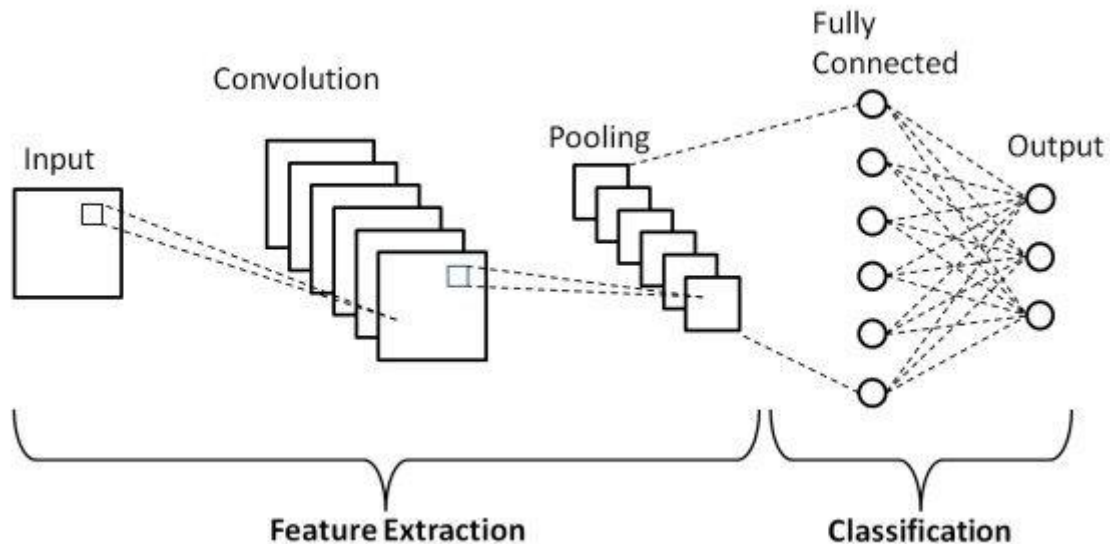
- **Purpose:** Converts the 2D feature maps into a 1D vector.
- **Why:** Fully connected layers that follow require 1D input.
- **Example:** A (7, 7, 64) feature map becomes a vector of size 3136.

6. Fully Connected (Dense) Layer

- **Purpose:** Performs final classification or prediction.
- **How it works:** Each neuron is connected to all outputs from the previous layer.

7. Output Layer

- **Purpose:** Produces the final prediction.
- **Activation Function:** Depends on the task:
 - **Softmax** → For multi-class classification.
 - **Sigmoid** → For binary classification.
 - **Linear** → For regression tasks.



Figure[15]: CNN Architecture used for pneumonia classification

2. Application

We build our **web application** using **Clean Architecture** and **Model-View-Intent (MVI)** principles.

Why move away from traditional code style and random design patterns?

Using a normal, unstructured code style or random architectural decisions might work for small, throwaway projects—but they often lead to:

- **Tight coupling** between layers (UI, database, logic)
- **Hard-to-test** components
- **Poor scalability**
- **Difficult maintenance**, especially as the app grows

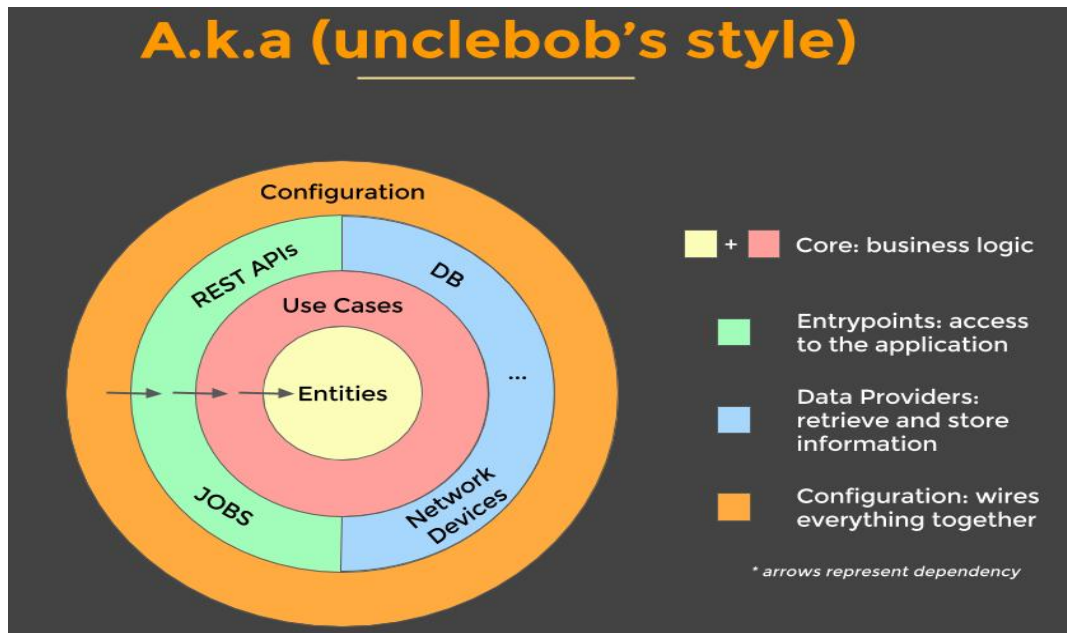
Clean Architecture and MVI provide a well-organized, testable, and maintainable structure that makes the system more **modular**, **independent**, and **future-proof**.

Clean Architecture for Web Applications:

Clean architecture is **crucial** for long-term, scalable web apps. It separates the system into layers, ensuring that:

- Business logic is isolated from frameworks
- UI and external APIs do not influence core logic
- The app is easy to test and evolve

This approach is particularly useful in team-based development and applications that must scale or live for years. .



Figure[16]: Clean Architecture was introduced by **Robert C. Martin (Uncle Bob)**

Explanation of Layers (for Web Architecture)

1. Entities (Enterprise Business Rules)

- **Location:** Core inner circle
- **Purpose:** Represent core business logic and data models (e.g., User, Order, Product)
- **Independence:** Must be completely independent from any frameworks, UI, or databases. They can be shared across multiple applications or services.

2. Use Cases (Application Business Rules)

- **Location:** Second layer from the center
- **Purpose:** Contain application-specific logic. These orchestrate workflows (e.g., “PlaceOrder”, “RegisterUser”) and coordinate between entities and external systems.
- **Independence:** Independent from the UI, databases, or third-party services. Focuses only on business processes.

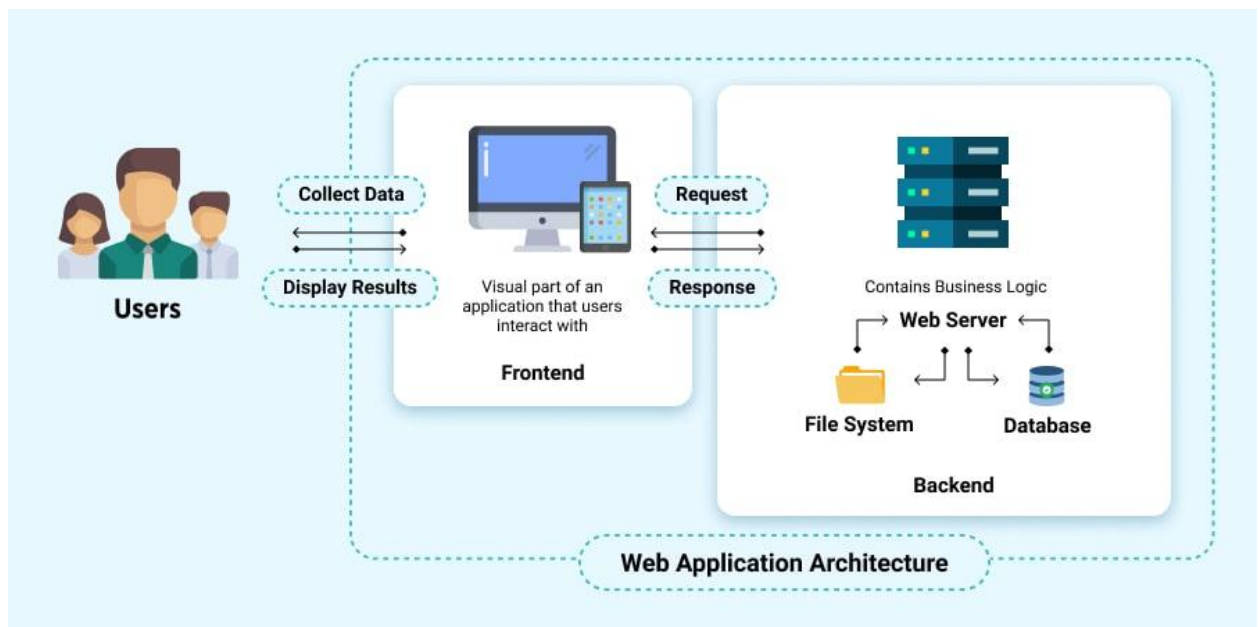
3. Interface Adapters

- **Location:** Surrounding use cases
- **Purpose:** Act as a bridge between external input/output and core logic. Responsible for transforming data between external representations (e.g., JSON over HTTP) and internal models.
- **Components:**
 - **Controllers** (handle web requests)

- **Presenters** (format response data)
- **Gateways/Repositories** (abstract data access)

4. Frameworks and Drivers

- **Location:** Outer layer
- **Purpose:** Represents external tools used by the application:
 - Web frameworks (e.g., FastAPI, ASP.NET Core)
 - Databases (PostgreSQL, MongoDB)
 - Authentication services, APIs, etc.
- **Independence:** These tools depend on the inner layers. The core of your app should not depend on any specific framework or database.



Figure[17]: Web application architecture

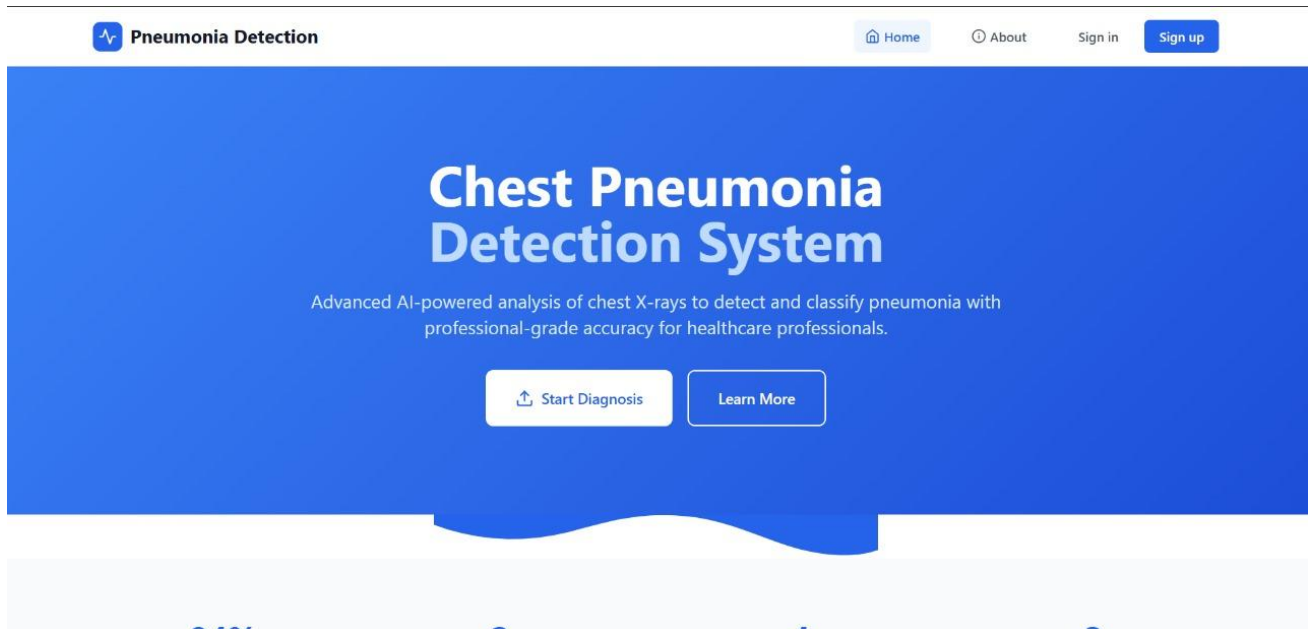
Why Should I Use Clean Architecture for Web Applications?

- **Maintainable and Expandable:** Clean architecture enables you to build modular web applications that are easy to maintain and scale as requirements grow.
- **Fits Well with Large Teams and Projects:** By clearly separating concerns, teams can work independently on different layers like frontend, backend, and APIs.
- **Testability:** Each layer is isolated, making it easier to write unit and integration tests for your web services, business logic, and components.
- **Flexibility and Adaptability:** Clean architecture supports changes in the tech stack (e.g., switching from REST to GraphQL) without impacting core business rules.

5.2 User Interface

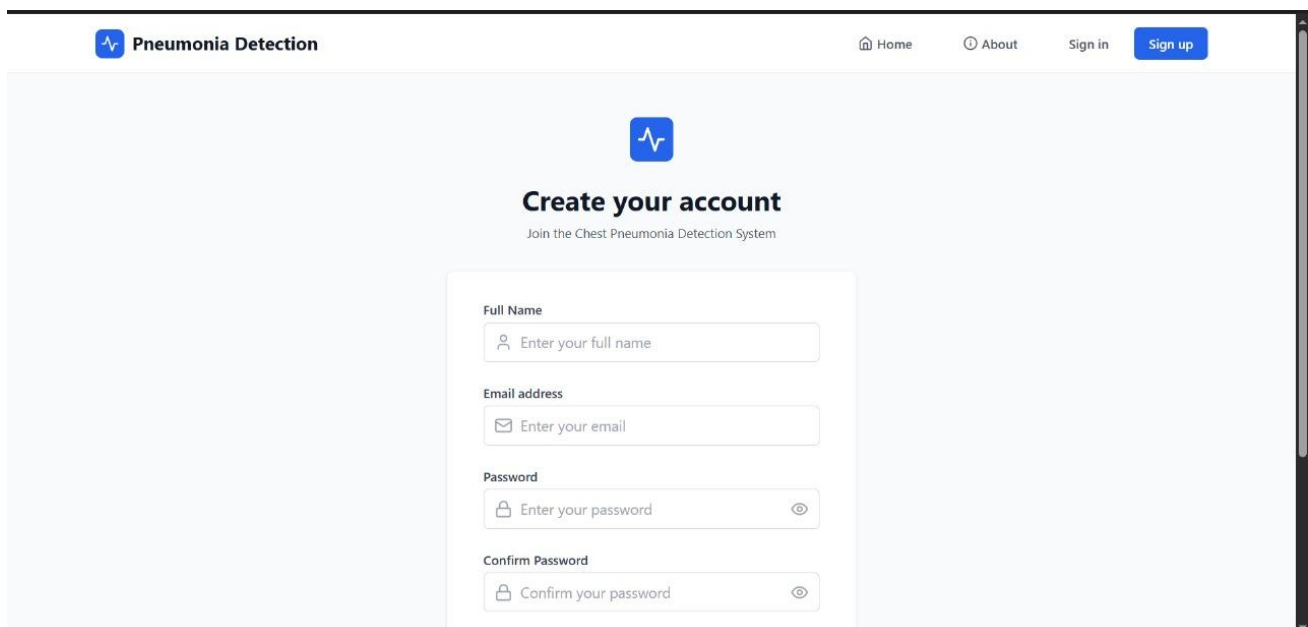
Our web application contains multiple screens, each screen has a state and action :

- 1- Home page: the user can click any button to move to the next page



Figure[18]: Home Page in web application

- 2- Sign-up page: the user can sign-up using username, e-mail and password



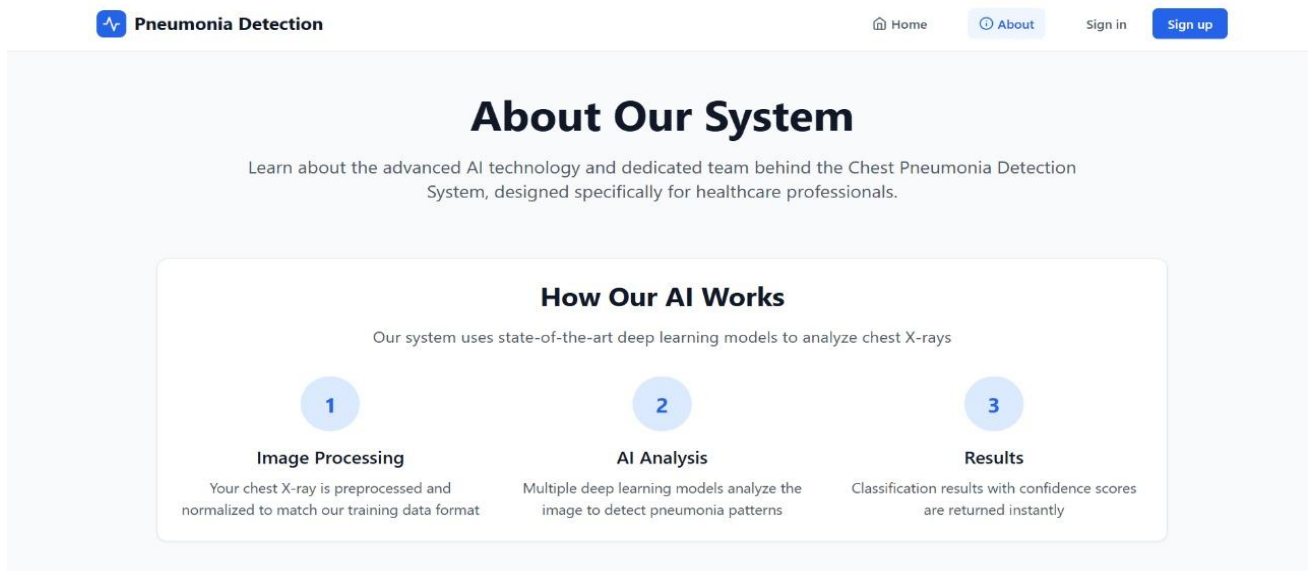
Figure[19]: Sign-up screen in web application

3- Sign-in: users can sign in the app using e-mail and password incase they have an account

The screenshot shows the 'Sign in to your account' page of the 'Pneumonia Detection' application. The page has a light blue header with the application logo and name on the left, and navigation links for 'Home', 'About', 'Sign in', and a 'Sign up' button on the right. The main content area features a central white box with the title 'Sign in to your account' and the subtitle 'Access the Chest Pneumonia Detection System'. Inside this box are two input fields: 'Email address' with a placeholder 'Enter your email' and 'Password' with a placeholder 'Enter your password' and a toggle eye icon. Below the fields is a blue 'Sign in' button. At the bottom of the box, there is a link that says 'Dont have an account? Sign up'.

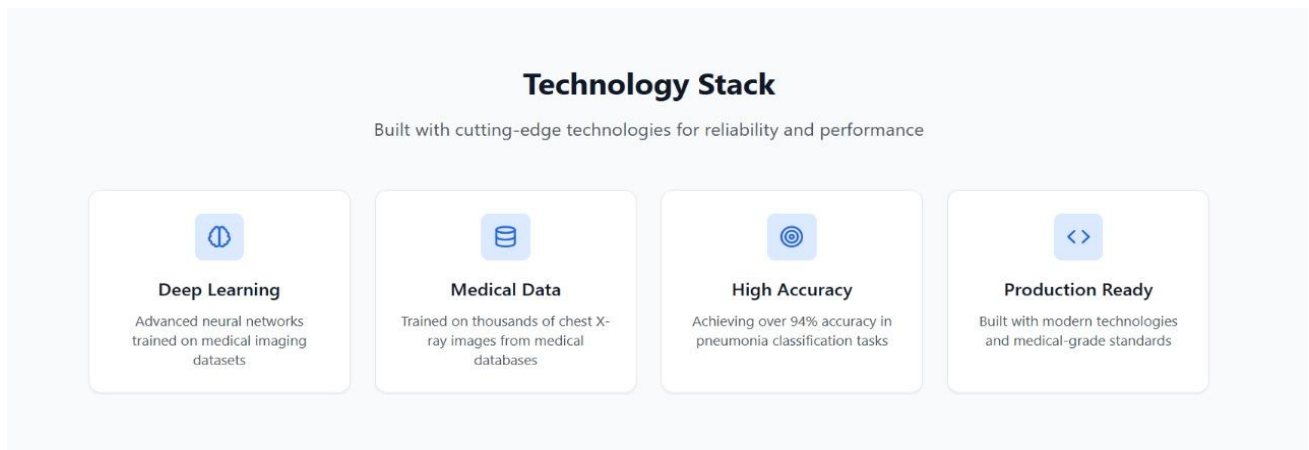
Figure[20]: Sign-in Page

4- About our system: this screen describes how the system works



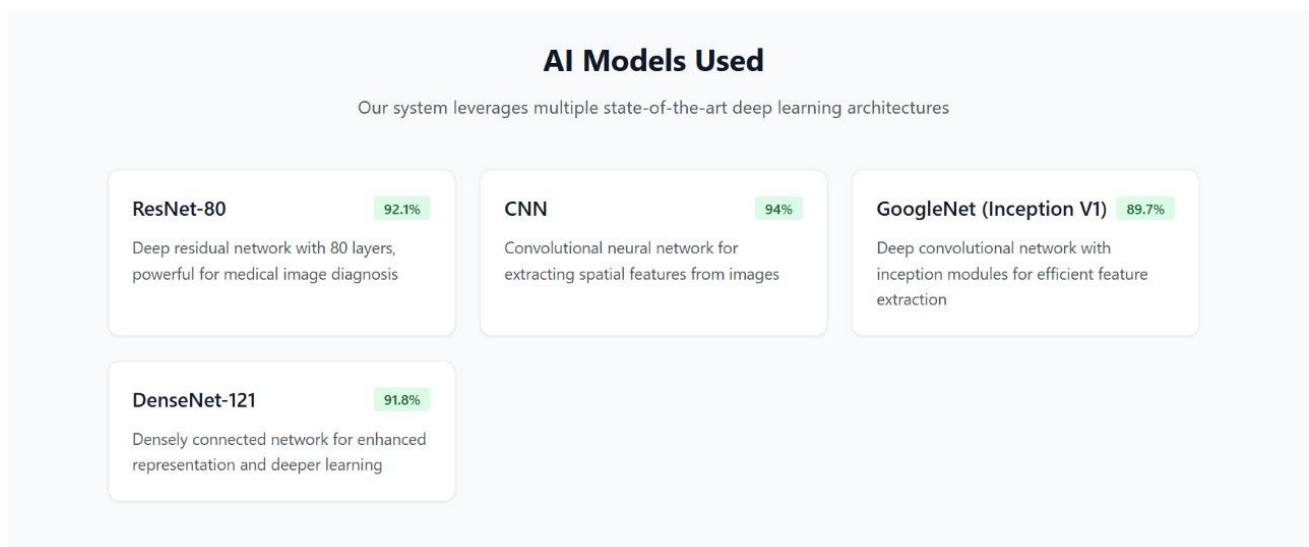
Figure[21]: About Our System Screen

5- Technology Stack



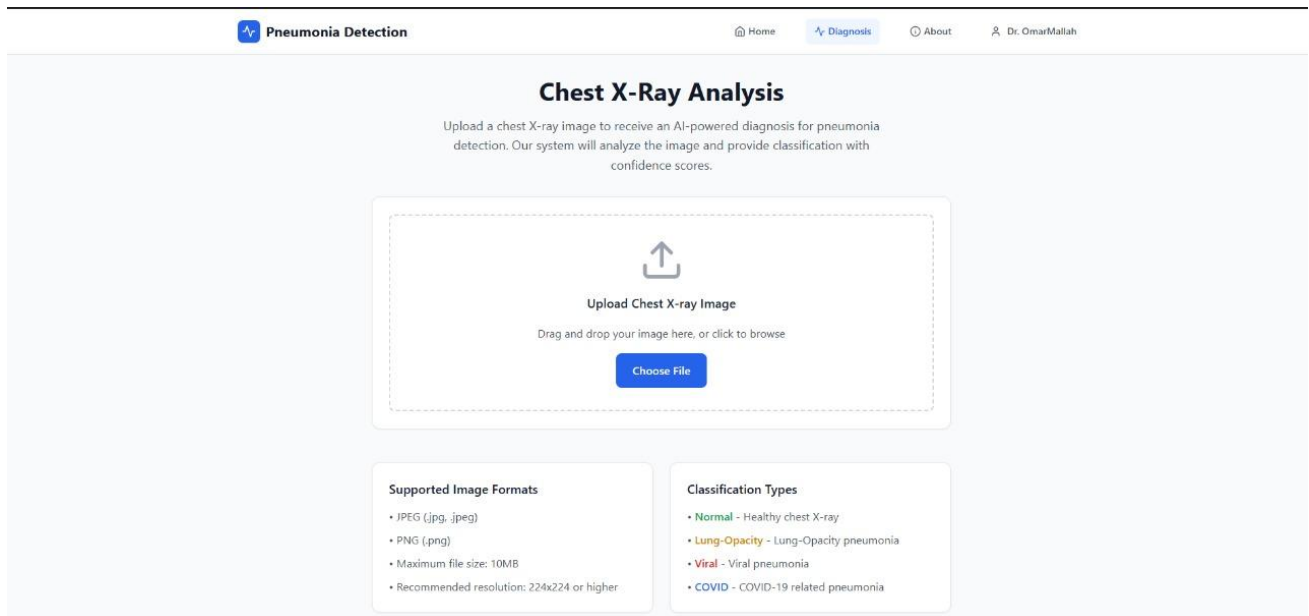
Figure[22]: Technology Stack Screen

6- AI Models used: this screen shows the AI models used for image classification where we Implemented the model with the highest accuracy (CNN model)



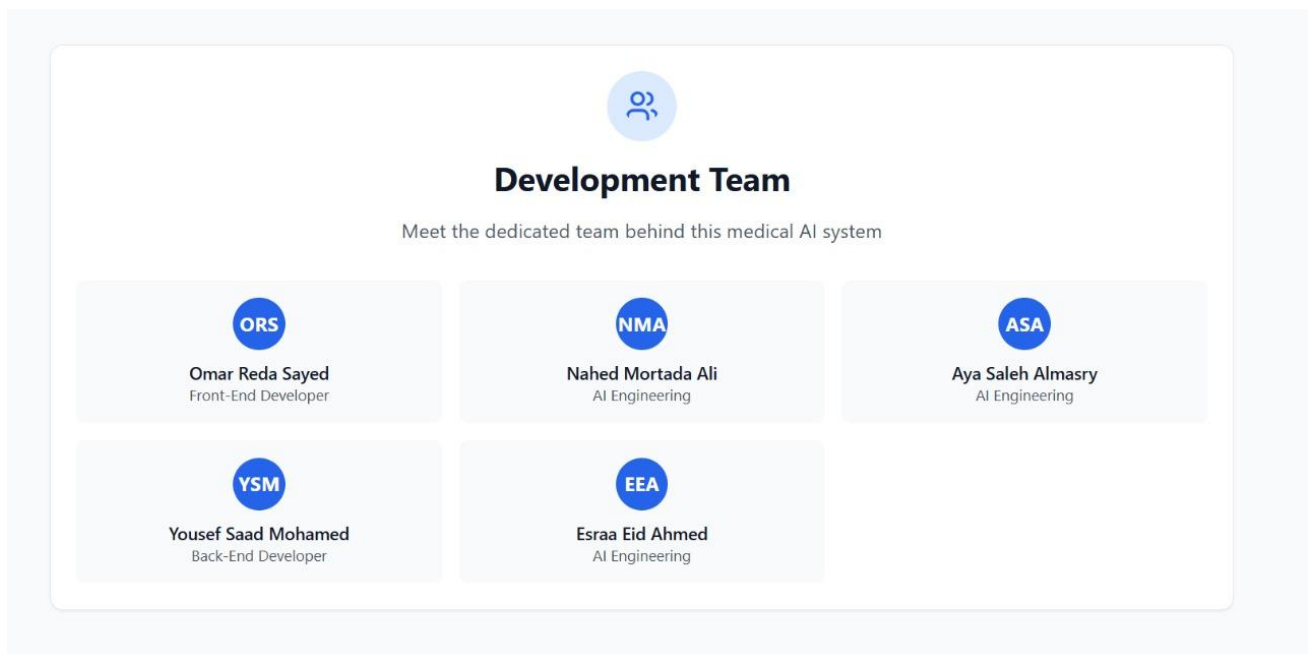
Figure[23]: About AI Models Screen

- 7- Chest X-ray analysis: this screen provides an choose file button to upload the image also the user can drag and drop the image directly



Figure[24]: Chest X-ray analysis screen

8- Development Team



Figure[25]: Development Team screen

5.3 Results and Discussion

Model Overview:

In this project, we developed a custom Convolutional Neural Network (CNN) model to classify chest X-ray images into four categories: Normal, COVID, Viral Pneumonia, and Lung Opacity. The CNN was built using the Keras Sequential API with layers such as Conv2D, MaxPooling2D, BatchNormalization, Dropout, and Dense. All input images were resized to 128×128 grayscale format to standardize dimensions and reduce computational load.

In addition to the CNN, we implemented an ensemble learning model that combines the predictions of multiple deep learning architectures to improve robustness and generalization. The ensemble included models such as ResNet18, DenseNet121, and GoogLeNet. These models were chosen due to their strong performance in image classification tasks and their complementary learning capabilities. The ensemble approach used majority voting and was trained on the same dataset as the CNN for comparative evaluation.

Training Details:

The CNN was trained using the Adam optimizer, which is well-suited for problems with sparse gradients and dynamic learning rates. We employed callbacks such as EarlyStopping and ReduceLROnPlateau to avoid overfitting and to fine-tune the training process. Image augmentation techniques like rotation, zooming, flipping, and shifting were applied to enrich the dataset and improve model generalizability.

The ensemble model was trained on the same preprocessed and augmented dataset. Each base learner contributed to the final prediction through a soft-voting mechanism. While the ensemble achieved decent performance, it did not surpass the CNN model.

Performance Metrics:

The CNN model achieved a training accuracy of 94% and a testing accuracy of 85%, as indicated by the classification report and confusion matrix. The ensemble model, by comparison, achieved slightly lower performance metrics, with training accuracy around 91% and testing accuracy at 83%.

The classification report for the CNN showed higher precision and recall values across most categories. The confusion matrix demonstrated a strong diagonal pattern, indicating that the CNN made accurate predictions with minimal confusion between classes.

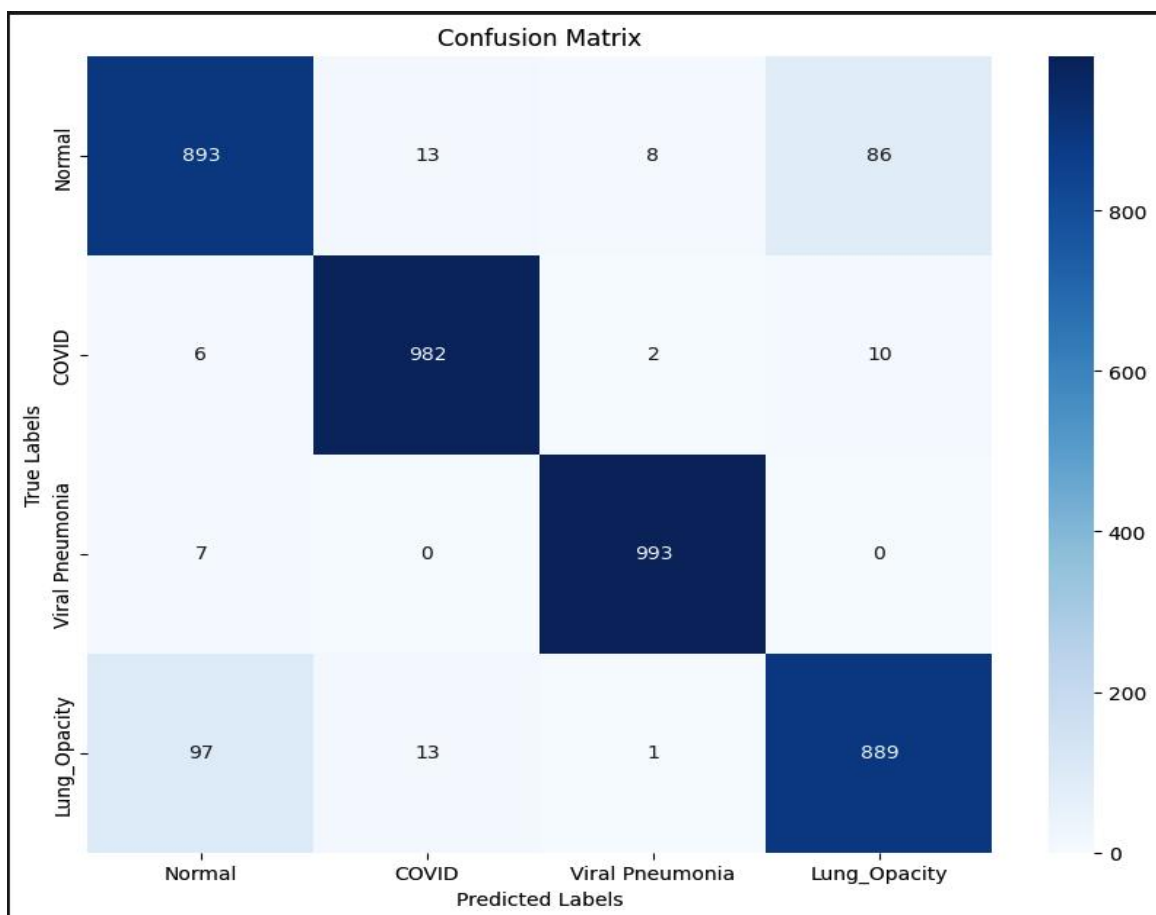
```

...
Classification Report:

```

	precision	recall	f1-score	support
Normal	0.89	0.89	0.89	1000
COVID	0.97	0.98	0.98	1000
Viral Pneumonia	0.99	0.99	0.99	1000
Lung_Opacity	0.90	0.89	0.90	1000
accuracy			0.94	4000
macro avg	0.94	0.94	0.94	4000
weighted avg	0.94	0.94	0.94	4000

Figure[26]: Classification Report of CNN Model



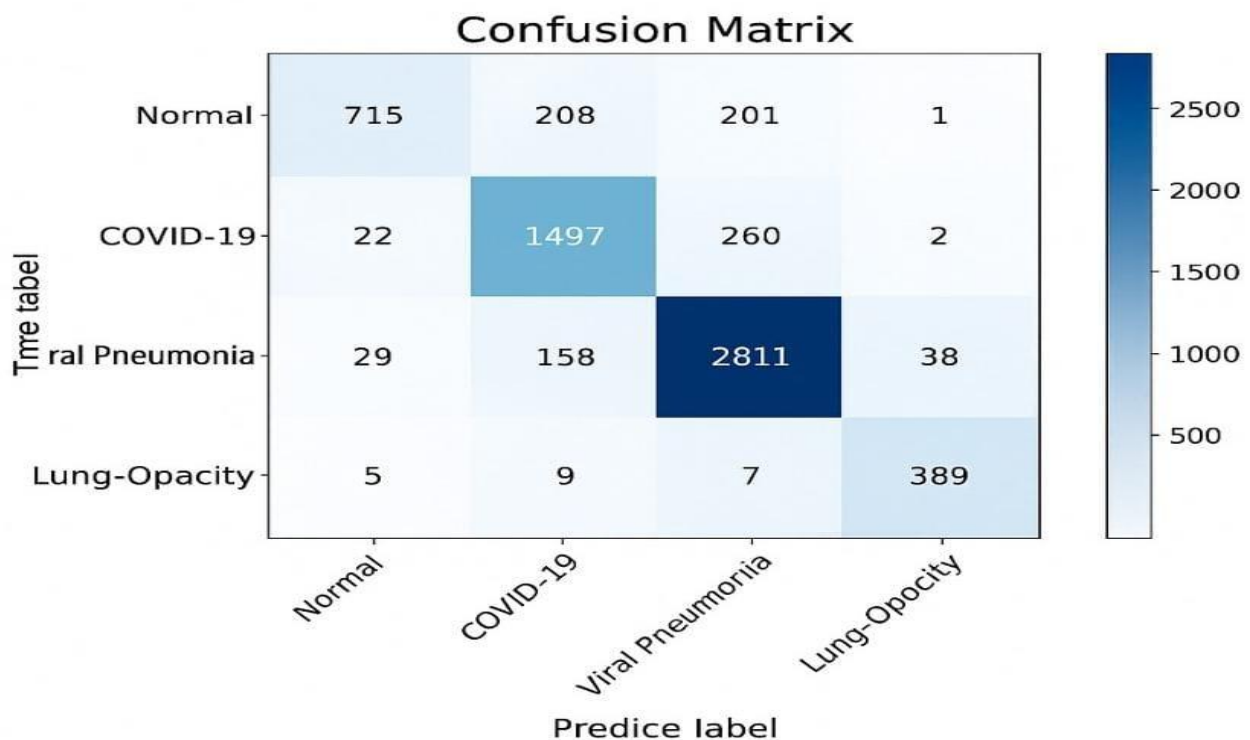
Figure[27]: Confusion Matrix of CNN Model

The ensemble model—combining ResNet18, DenseNet121, and GoogLeNet using majority voting—achieved:

- **Training Accuracy: up to 99.47%**
- **Testing Accuracy: 85.27%**
- More balanced performance across some classes, but slightly lower precision in key classes compared to the CNN.

Classification Report:				
	precision	recall	f1-score	suport
Normal	0.93	0.64	0.75	1125
COVID-19	0.80	0.84	0.82	1781
Viral Pneumonia	0.86	0.93	0.89	3036
Lung-Opacity	0.90	0.95	0.93	410
accuracy	0.87	0.84	0.85	6352
macro avg	0.87	0.84	0.85	6352
weighted avg	0.86	0.85	0.85	6352

Figure[28]: classification report of ensemble model

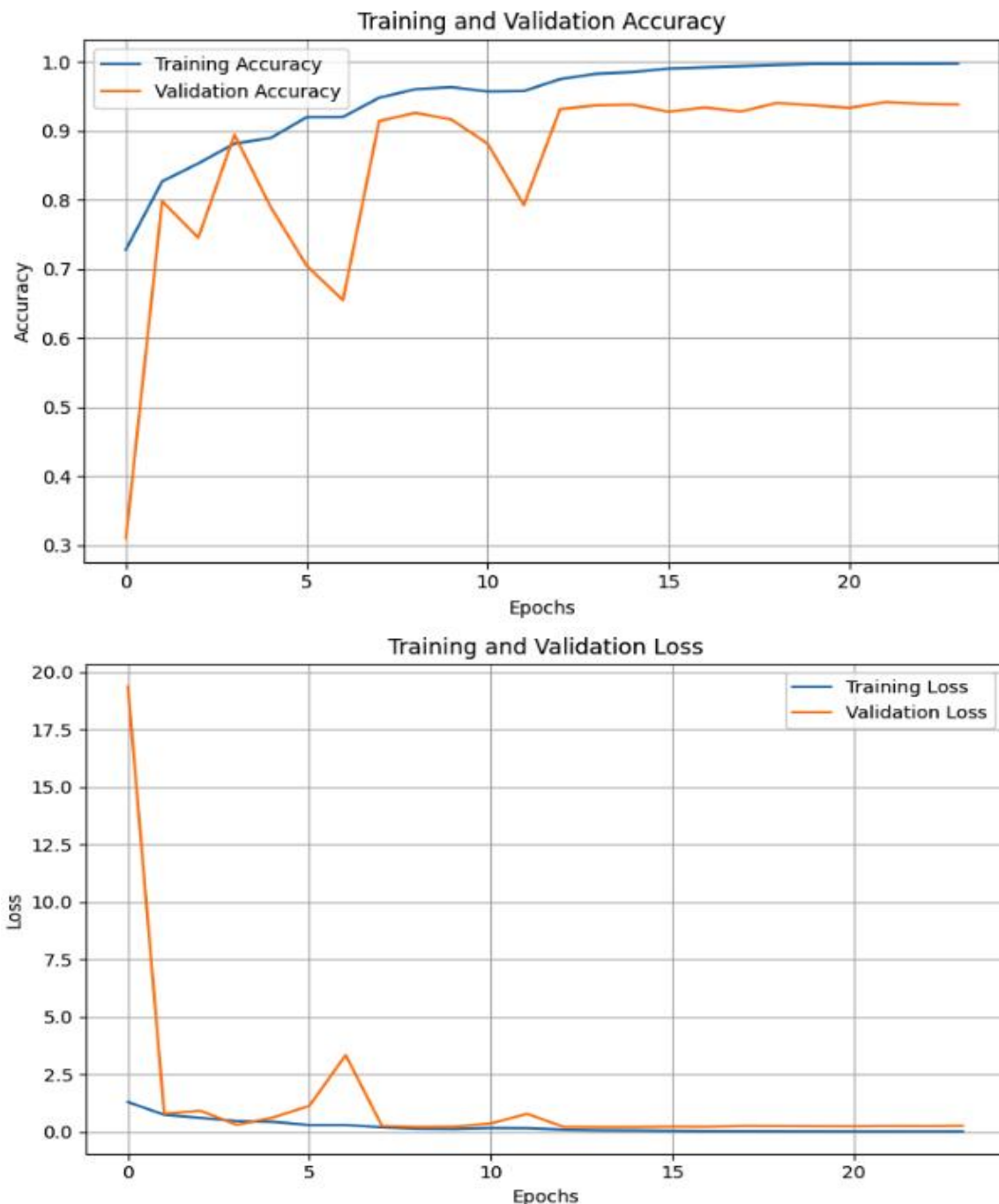


Figure[29]: Confusion matrix of ensemble model

Learning Curves:

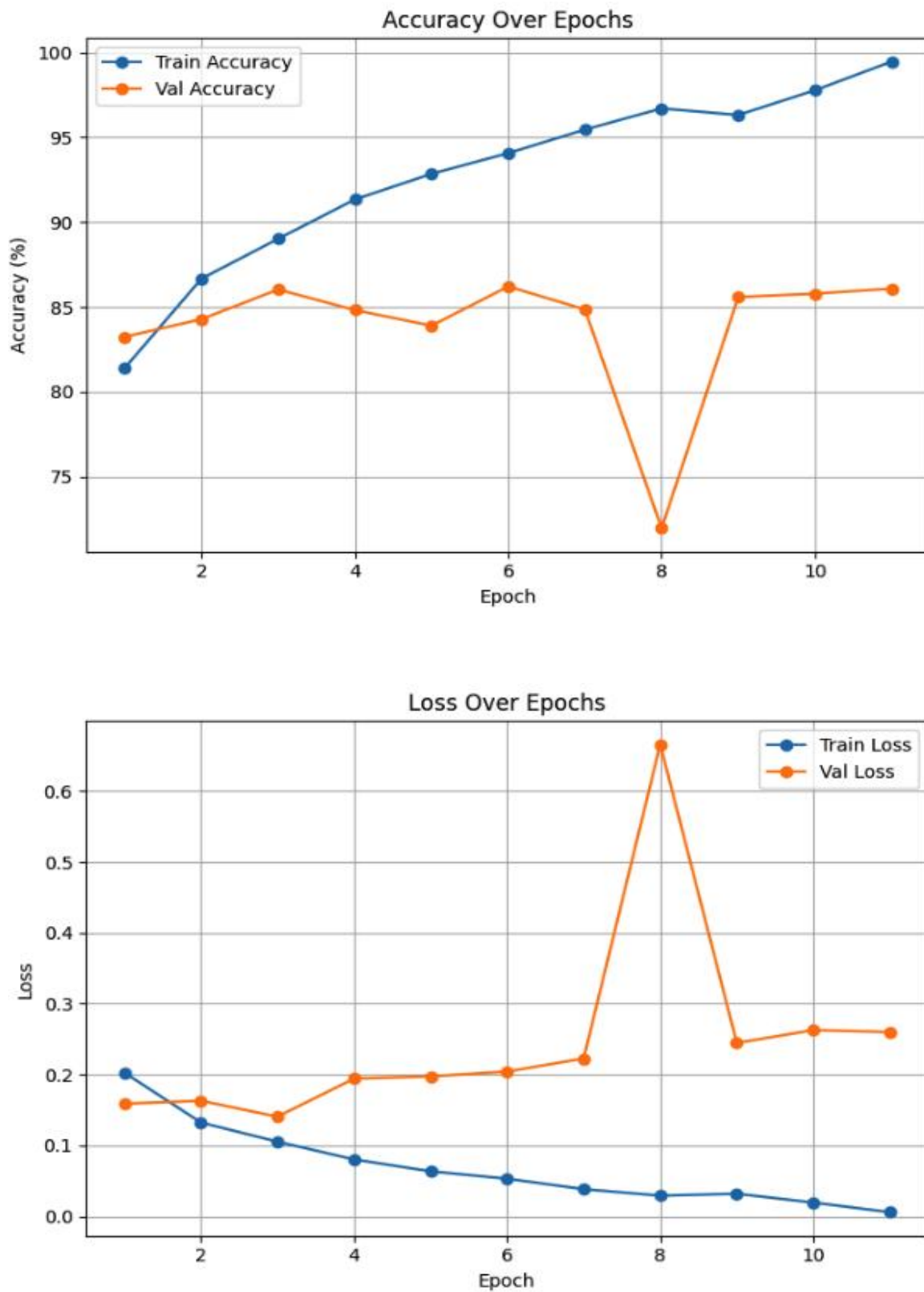
The CNN's training and validation accuracy and loss curves over 25 epochs showed:

- Rapid convergence in the early epochs.
- Smooth and consistent improvement without significant overfitting.
- Efficient reduction in training and validation loss.



Figure[30]: Training and Validation Accuracy and Loss Over Epochs of CNN model

The ensemble model's learning pattern was less smooth, likely due to the limitations of combining base learners that are not as deeply trained on complex image features as CNNs.



Figure[31]: Training and Validation Accuracy and Loss Over Epochs of ensemble model

Both models performed well, but the CNN consistently demonstrated better classification performance and generalization.

While ensemble learning introduced an additional comparative approach, the CNN model outperformed it in terms of both accuracy and stability. The deeper architecture and end-to-end feature extraction capabilities of the CNN allowed it to learn more discriminative features from chest X-ray images, making it the preferred model for this task.

Chapter6: Testing

6.1 Unit Testing in Web APIs

1. Introduction to Unit Testing

Unit testing is a foundational software engineering practice that focuses on testing individual components—such as methods, functions, or classes—in isolation. In web API development (whether using .NET, FastAPI, Django, Flask, Node.js, etc.), unit tests validate backend logic independently from external systems like databases, APIs, or cloud services. A solid unit testing strategy increases confidence in the codebase and helps maintain reliability over time.

2. Purpose of Unit Testing in the Project

The key purposes of unit testing in a web API project include:

- **Logic Validation:** Confirming that each method, service, or function behaves as expected for given inputs.
- **Early Bug Detection:** Identifying defects during development before code is deployed or integrated.
- **Component Isolation:** Using mocks or stubs to isolate dependencies (e.g., databases, APIs, message queues).
- **Refactoring Safety:** Supporting code changes and improvements without unintentionally breaking functionality.

3. Overview of Current Testing Approach

In our current implementation, the testing approach emphasizes:

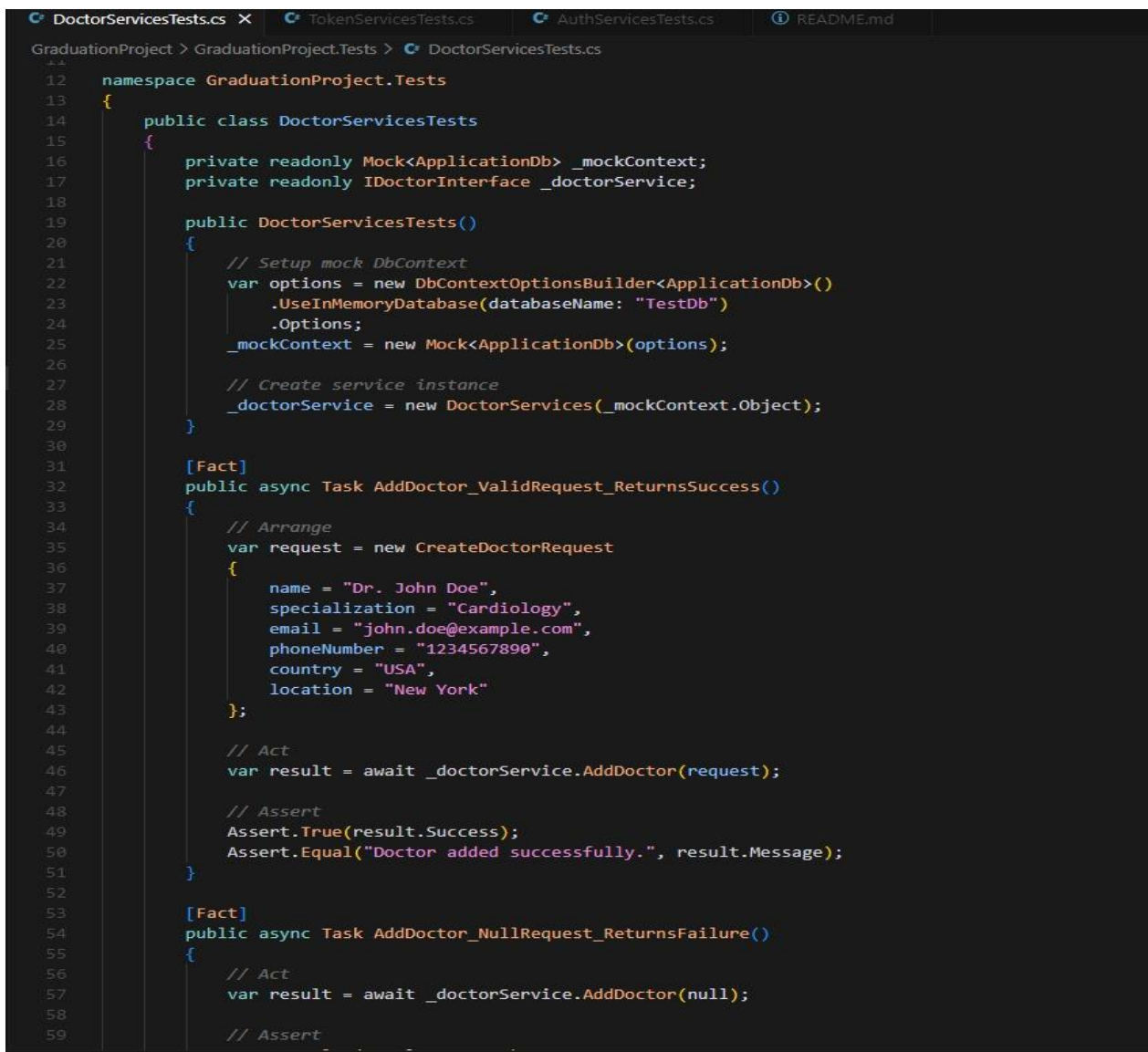
- Testing services or business logic layers independently of infrastructure or I/O.
- Mocking repositories, API clients, or external services to focus tests on logic, not integration.
- Handling edge cases, such as null inputs, boundary values, or error conditions.
- Running tests with frameworks such as xUnit or NUnit (.NET), pytest (Python), Jest (Node.js), or others.

These tests provide fast feedback and help enforce application correctness during active development.

4. Future Testing Enhancements

To improve test coverage and overall system reliability, the following enhancements are planned:

- **Endpoint/Controller Testing:** Validating API routes/controllers using mocked or in-memory service layers.
- **Input Validation & Error Handling Tests:** Ensuring that incorrect or malformed inputs are properly handled with meaningful error responses.
- **Integration Testing:** Introducing real dependencies (e.g., test databases, APIs) to test multiple components together.
- **Coverage Expansion:** Including more scenarios across modules, such as authentication, file handling, concurrency, and edge cases.



```
12 namespace GraduationProject.Tests
13 {
14     public class DoctorServicesTests
15     {
16         private readonly Mock<ApplicationDb> _mockContext;
17         private readonly IDoctorInterface _doctorService;
18
19         public DoctorServicesTests()
20         {
21             // Setup mock DbContext
22             var options = new DbContextOptionsBuilder<ApplicationDb>()
23                 .UseInMemoryDatabase(databaseName: "TestDb")
24                 .Options;
25             _mockContext = new Mock<ApplicationDb>(options);
26
27             // Create service instance
28             _doctorService = new DoctorServices(_mockContext.Object);
29         }
30
31         [Fact]
32         public async Task AddDoctor_ValidRequest_ReturnsSuccess()
33         {
34             // Arrange
35             var request = new CreateDoctorRequest
36             {
37                 name = "Dr. John Doe",
38                 specialization = "Cardiology",
39                 email = "john.doe@example.com",
40                 phoneNumber = "1234567890",
41                 country = "USA",
42                 location = "New York"
43             };
44
45             // Act
46             var result = await _doctorService.AddDoctor(request);
47
48             // Assert
49             Assert.True(result.Success);
50             Assert.Equal("Doctor added successfully.", result.Message);
51         }
52
53         [Fact]
54         public async Task AddDoctor_NullRequest_ReturnsFailure()
55         {
56             // Act
57             var result = await _doctorService.AddDoctor(null);
58
59             // Assert
```

Figure[32]: Doctor service tests in .NET API

```

6  using System.Collections.Generic;
7  using Xunit;
8
9  namespace GraduationProject.Tests
10 {
11     public class TokenServicesTests
12     {
13         private readonly Mock<IConfiguration> _mockConfiguration;
14         private readonly ITokenServices _tokenService;
15
16         public TokenServicesTests()
17         {
18             // Setup configuration mock
19             _mockConfiguration = new Mock<IConfiguration>();
20             _mockConfiguration.Setup(x => x["JWT:Key"]).Returns("YourSecretKeyHere12345678901234567890");
21             _mockConfiguration.Setup(x => x["JWT:Issuer"]).Returns("TestIssuer");
22             _mockConfiguration.Setup(x => x["JWT:Audience"]).Returns("TestAudience");
23
24             // Create service instance
25             _tokenService = new TokenServices(_mockConfiguration.Object);
26         }
27
28         [Fact]
29         public void CreateToken_ValidUser_ReturnsToken()
30         {
31             // Arrange
32             var user = new User
33             {
34                 UserName = "testuser",
35                 Email = "test@example.com"
36             };
37
38             // Act
39             var token = _tokenService.CreateToken(user);
40
41             // Assert
42             Assert.NotNull(token);
43             Assert.NotEmpty(token);
44         }
45
46         [Fact]
47         public void CreateToken_NullUser_ThrowsArgumentNullException()
48         {
49             // Act & Assert
50             Assert.Throws<ArgumentNullException>(() => _tokenService.CreateToken(null));
51         }
52
53         [Fact]
54         public void CreateToken_ValidUser_ContainsUserClaims()

```

Figure[33]: Token service tests in .NET API

```

GraduationProject > GraduationProject.Tests > AuthServicesTests.cs
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.Logging;
using Microsoft.Extensions.Options;
using Moq;
using GraduationProject.Models;
using GraduationProject.Services;
using GraduationProject.Tests;

namespace GraduationProject.Tests
{
    public class AuthServicesTests
    {
        private readonly Mock<UserManager<User>> _mockUserManager;
        private readonly Mock<SignInManager<User>> _mockSignInManager;
        private readonly IAuthServices _authService;

        public AuthServicesTests()
        {
            // Setup UserManager mock
            var userStoreMock = new Mock<IUserStore<User>>();
            _mockUserManager = new Mock<UserManager<User>>(
                userStoreMock.Object,
                null, null, null, null, null, null, null);

            // Setup SignInManager mock
            _mockSignInManager = new Mock<SignInManager<User>>(
                _mockUserManager.Object,
                Mock.Of<Microsoft.AspNetCore.Http.IHttpContextAccessor>(),
                Mock.Of<IUserClaimsPrincipalFactory<User>>(),
                null, null, null, null);

            // Create service instance
            _authService = new AuthServices(_mockUserManager.Object, _mockSignInManager.Object);

            [Fact]
            public async Task Register_ValidUser_ReturnsSuccess()
            {
                // Arrange
                var userRequest = new UserRequest
                {
                    userName = "testuser",
                    email = "test@example.com",
                    password = "Test123!"
                };

                _mockUserManager.Setup(x => x.CreateAsync(It.IsAny<User>(), It.IsAny<string>()))
                    .ReturnsAsync(IdentityResult.Success);

                // Act
                var result = await _authService.Register(userRequest);

                // Assert
                Assert.True(result.Succeeded);
            }
        }
    }
}

```

Figure[34]: Authentication service tests in .NET API

Response body

```

{
  "predicted_class": "Viral Pneumonia",
  "confidence": 0.9999090433120728,
  "all_predictions": {
    "COVID": 1.9587163890832926e-8,
    "Lung Opacity": 8.896944905245618e-7,
    "Normal": 0.00009014001989271492,
    "Viral Pneumonia": 0.9999090433120728
  }
}

```

Figure[35]: Testing a Viral pneumonia diagnosed chest X-ray in fastAPI

Response body

```
{
  "predicted_class": "Normal",
  "confidence": 0.9999974966049194,
  "all_predictions": {
    "COVID": 2.2387948699770233e-12,
    "Lung Opacity": 0.000002500466735000373,
    "Normal": 0.9999974966049194,
    "Viral Pneumonia": 1.8129607537442638e-10
  }
}
```

Figure[36]: Testing a Normal diagnosed chest X-ray in fastAPI

Response body

```
{
  "predicted_class": "COVID",
  "confidence": 0.99946528673172,
  "all_predictions": {
    "COVID": 0.99946528673172,
    "Lung Opacity": 3.426829664476827e-7,
    "Normal": 0.0005294143338687718,
    "Viral Pneumonia": 0.000005001078079658328
  }
}
```

Figure[37]: Testing a Normal diagnosed chest X-ray in fastAPI

Response body

```
{
  "predicted_class": "Lung Opacity",
  "confidence": 0.9999334812164307,
  "all_predictions": {
    "COVID": 0.000018279006326338276,
    "Lung Opacity": 0.9999334812164307,
    "Normal": 0.000007715454557910562,
    "Viral Pneumonia": 0.000040569648263044655
  }
}
```

Figure[38]: Testing a Lung-Opacity diagnosed chest X-ray in fastAPI

1. Application:

We have thoroughly tested each screen within the app and switched between screens to ensure there are no errors in the user interface.

2. Server:

Application to Server and Server to Application:

We have tested the communication between the application and the server to ensure it is error-free.

This involves:

- Accurately receiving images on the server from the application.
- Correctly sending and receiving information between the server and the application.
- Ensuring the speed of data collection and transmission between the application and the server is optimal.

3. Model:

We ensured the chest pneumonia model does not have overfitting, The results clearly indicated that everything is functioning correctly.

We used manual testing and here were the test cases:

Project Name	Multi Classification Chest X-Ray using Deep Learning
Priority	High

Test Case Author	Aya Saleh, Nahed Mortada and Esraa Eid
Test Case Reviewers	Whole Team

Test Case ID	Test Scenario	Test Case	Test Steps	Expected Results	Actual Results	Status
TC_SIGNUP_002	Verify the Sign-up	Enter Valid Information	1-Enter Username 2-Enter Email 3-Enter Password 4-Click on Sign-up	Successful Sign-up	As Expected	Success
TC_SIGNUP_002	Verify the Sign-up	Enter an Email already used before	1-Enter Username 2-Enter Email 3-Enter Password 3-Click on Sign-up	A message "the Email already in use"	As Expected	Success
TC_SIGNUP_002	Verify the Sign-up	Enter weak password	1-Enter Username 2-Enter Email 3-Enter Password 3-Click on Sign-up	A message "Your password must contain capital and small letters & atleast 8 characters"	As Expected	Success
TC_SIGNUP_002	Verify the Sign-up	Enter Ivalid Email	1-Enter Username 2-Enter Email 3-Enter Password 3-Click on Sign-up	A message "Pl ease enter valid Email"	As Expected	Success

Table[14]: Verify Sign-up of Users

Test Case ID	Test Scenario	Test Case	Test Steps	Expected Results	Actual Results	Status
TC_LOGIN_003	Verify the Login	Enter Valid Email& Valid Password	1-Enter Email 2-Enter Password 3-Click on Login	Successful Login	As Expected	Success
TC_LOGIN_003	Verify the Login	Enter Valid Email& Invalid Password	1-Enter Email 2-Enter Password 3-Click on Login	A message "Wrong Password"	As Expected	Success
TC_LOGIN_003	Verify the Login	Enter Invalid Email& Valid Password	1-Enter Email 2-Enter Password 3-Click on Login	A message "User Not Found"	As Expected	Success
TC_LOGIN_003	Verify the Login	Enter Invalid Email& Invalid Password	1-Enter Email 2-Enter Password 3-Click on Login	A message "You have to check your Email& Password"	As Expected	Success

Table[15]: Verify Log-in of users

Test Case ID	Test Scenario	Test Case	Test Steps	Expected Results	Actual Results	Status
TC_IMAGE_DIAGNOSIS_004	Verify The Image Diagnosis	Upload Valid Image	1-Upload Image 2-Click on detect	Return the Chest pneumonia type	As Expected	Success
TC_IMAGE_DIAGNOSIS_004	Verify The Image Diagnosis	Upload an Image that's not a chest X-ray	1-Upload Image 2-Click on detect	Return Error Message	Return Random type	Failed

Table[16]: Verify The Image Diagnosis

Test Case ID	Test Scenario	Test Case	Test Steps	Expected Results	Actual Results	Status
TC_LOGOUT_005	Verify Log-out	Click on Log out	1-Click on Log out	Log out and redirect to welcome screen	As Expected	Success

Table[17]: Verify Log-Out of users

6.2 Integration Testing

Integration testing is a level of software testing where individual components of a software system are combined and tested as a group. In our project, this involved combining the various modules of the chest X-ray classification system—such as data preprocessing, model inference, and result visualization—into a unified pipeline.

The goal of this testing phase was to ensure that all components work together correctly, the data flows seamlessly between them, and the overall system performs reliably under different usage scenarios.

Conclusion

Our investigation revealed that maintaining respiratory health is one of the most pressing global health challenges. In light of this, we focused on detecting and classifying chest X-ray images, particularly targeting COVID-19, viral pneumonia, and normal lung conditions using deep learning.

Early and accurate diagnosis of chest-related conditions—especially during pandemics—is critical to prevent misdiagnosis, reduce the burden on healthcare systems, and provide timely treatment. Delay in diagnosis can result in worsening of symptoms, longer hospitalization periods, and even fatal complications. Hence, reliable chest X-ray classification plays a vital role in aiding doctors and improving patient outcomes.

This project proposes a machine learning-based approach for automatic classification of chest X-ray images. This tool is intended to support medical professionals by reducing diagnosis time and enhancing accuracy, while also enabling the potential for self-assessment tools for public use.

Initially, we explored building an ensemble model using multiple deep learning architectures. However, the ensemble approach did not produce satisfactory results in terms of accuracy. After further experimentation, we shifted our focus to Convolutional Neural Networks (CNN), which delivered significantly better performance.

We used the COVID-19 Radiography Database from Kaggle ([link](#)), which includes a diverse set of labeled chest X-ray images across categories such as COVID-19, Viral Pneumonia, and Normal.

After applying preprocessing and data augmentation techniques (rotation, zoom, and flipping), the dataset contained approximately 34,000 images. We divided the data into 80% for training, 10% for validation, and 10% for testing.

We used a CNN model with optimized hyperparameters and the Adam optimizer (learning rate = 0.0001). After several iterations, the model achieved an accuracy of 94%, significantly outperforming our earlier ensemble attempts.

Future Work

To improve and expand the chest X-ray classification system, we plan to include the following enhancements:

- **Data Expansion:** Incorporate more diverse chest X-ray datasets (e.g., tuberculosis, lung opacity) to enhance generalizability.
- **Accuracy Improvement:** Continue refining the CNN architecture and explore transfer learning models to boost performance beyond 94%.
- **Broader Conditions Detection:** Extend the system to detect additional thoracic diseases beyond COVID-19 and pneumonia.
- **Language Support:** Integrate multilingual chatbot capabilities to serve non-English speaking populations.
- **Deployment & Market Expansion:** Initially collaborate with healthcare providers and radiology centers for professional use, followed by a broader public release.
- **Enhanced Chatbot Features:** Allow users to upload X-ray images and receive chatbot-generated feedback, including summarization of doctor notes.
- **Sound Speaker Integration:** Implement an audio feature that reads out the predicted diagnosis, improving accessibility for visually impaired users.

By implementing these features, we aim to deliver a more comprehensive and accessible diagnostic tool for chest-related diseases using medical imaging.

References:

1. Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach*. Pearson.
2. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
3. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning*. Springer.
4. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*. Springer.
5. Breiman, L., Friedman, J., Stone, C. J., & Olshen, R. A. (1984). *Classification and Regression Trees*. CRC Press.
6. Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.
7. Cortes, C., & Vapnik, V. (1995). Support-Vector Networks. *Machine Learning*, 20(3), 273-297.
8. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
9. MacQueen, J. (1967). Some Methods for Classification and Analysis of Multivariate Observations. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*.
10. Johnson, S. C. (1967). Hierarchical Clustering Schemes. *Psychometrika*, 32(3), 241-254.
11. Jolliffe, I. T. (2002). *Principal Component Analysis*. Springer.
12. Hinton, G. E., & Salakhutdinov, R. R. (2006). Reducing the Dimensionality of Data with Neural Networks. *Science*, 313(5786), 504-507.
13. Zhu, X., & Goldberg, A. B. (2009). Introduction to Semi-Supervised Learning. *Synthesis Lectures on Artificial Intelligence and Machine Learning*, 3(1), 1-130.
14. Watkins, C. J., & Dayan, P. (1992). Q-Learning. *Machine Learning*, 8(3-4), 279-292.
15. Mnih, V., et al. (2015). Human-Level Control Through Deep Reinforcement Learning. *Nature*, 518(7540), 529-533.
16. Sutton, R. S., McAllester, D. A., Singh, S. P., & Mansour, Y. (2000). Policy Gradient Methods for Reinforcement Learning with Function Approximation. *Advances in Neural Information Processing Systems (NeurIPS)*.
17. LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep Learning. *Nature*, 521(7553), 436-444.

18. Schmidhuber, J. (2015). Deep Learning in Neural Networks: An Overview. *Neural Networks*, 61, 85-117.
19. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
20. LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep Learning. *Nature*, 521(7553), 436-444.
21. LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.
22. Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735-1780.
23. Vaswani, A., et al. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems (NeurIPS)*.
24. Hinton, G. E., & Salakhutdinov, R. R. (2006). Reducing the Dimensionality of Data with Neural Networks. *Science*, 313(5786), 504-507.
25. Goodfellow, I., et al. (2014). Generative Adversarial Networks. *Advances in Neural Information Processing Systems (NeurIPS)*.
26. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
27. LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep Learning. *Nature*, 521(7553), 436-444.
28. Rajpurkar, P., et al. (2017). CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning. *arXiv preprint arXiv:1711.05225*.
29. Tortora, G. J., & Derrickson, B. (2017). *Principles of Anatomy and Physiology*. Wiley.
30. World Health Organization (2023). Pneumonia. Retrieved from <https://www.who.int>.
31. Wang, X., et al. (2017). ChestX-ray8: Hospital-Scale Chest X-Ray Database and Benchmarks on Weakly-Supervised Classification and Localization of Common Thorax Diseases. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
32. Chowdhury, M. E., et al. (2020). Can AI Help in Screening Viral and COVID-19 Pneumonia? *IEEE Access*, 8, 132665-132676.
33. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
34. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

35. Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *Proceedings of the International Conference on Machine Learning (ICML)*.
36. Rajpurkar, P., et al. (2017). CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning. *arXiv preprint arXiv:1711.05225*.
37. Selvaraju, R. R., et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*.
38. Selvaraju, R. R., et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*.
39. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
40. Rajpurkar, P., et al. (2017). CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning. *arXiv preprint arXiv:1711.05225*.
41. <https://www.mdpi.com/2076-3417/12/13/6448>
42. https://thesai.org/Downloads/Volume15No6/Paper_104
[Classification of Pneumonia from Chest X ray Images.pdf](#)
43. <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0256630>
44. <https://www.iieta.org/journals/ts/paper/10.18280/ts.400317>
45. <https://www.iieta.org/journals/ts/paper/10.18280/ts.400113>